

Diseño de Sistemas

Escala un sistema completo con Flutter + Supabase: estimación, modelado de datos, cache, realtime, alta disponibilidad, seguridad y observabilidad. Con plantilla reutilizable y 4 casos integradores.

ENFOQUE

**System Design Primer
+ DDIA + Supabase**

STACK

**Flutter + Supabase
(Postgres, Realtime,
Edge)**

NIVEL

Intermedio a Avanzado

TIEMPO ESTIMADO

25-35 horas

Escalabilidad

Cache

Realtime

CAP

Contenido

19 capítulos · Diseño de Sistemas a escala con Flutter + Supabase

PARTE 1 · FUNDAMENTOS Y ESCALA

- 00 00: Fundamentos de System Design
- 01 01: Clarificar Requisitos
- 02 02: Estimación de Escala (Back-of-the-Envelope)
- 03 03: Componentes de Arquitectura

PARTE 2 · DATOS Y RENDIMIENTO

- 04 04: Modelado de Datos a Escala
- 05 05: Cache y Rendimiento

PARTE 3 · TIEMPO REAL Y ESCALABILIDAD

- 06 06: Realtime y Streaming
- 07 07: Escalabilidad y Alta Disponibilidad

PARTE 4 · SEGURIDAD Y OPERACIÓN

- 08 08: Seguridad en el Diseño
- 09 09: Observabilidad, Monitoreo y Resiliencia
- 10 10: Offline-First y Sincronización

PARTE 5 · PLANTILLA Y CASOS INTEGRADORES

- 11 11: Plantilla de Diseño de Sistema
- 12 12: Caso de Estudio — Feed de Red Social
- 13 13: Caso de Estudio — Chat / Mensajería
- 14 14: Caso de Estudio — E-Commerce Escalable

15 **15: Caso de Estudio — SaaS Multi-Tenant**

PARTE 6 · PRÁCTICA Y CIERRE

16 **16: Ejercicios de Práctica**

17 **17: Recursos Externos**

BI **Bibliografía**

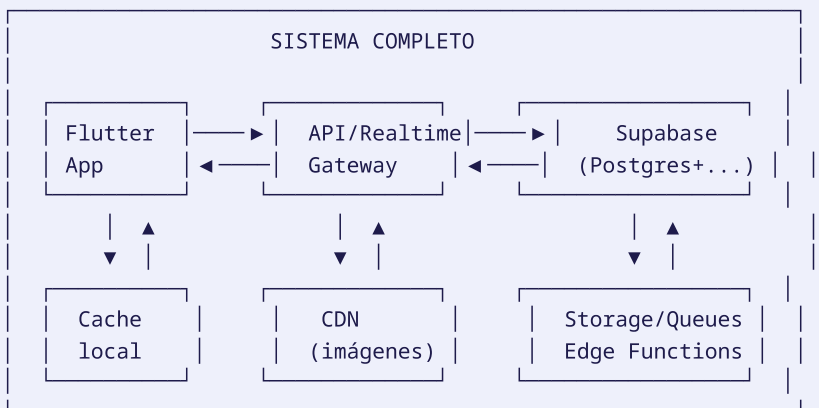
00: Fundamentos de System Design

System Design es el proceso de definir la **arquitectura, las interfaces y los datos** de un sistema completo para que satisfaga requisitos funcionales y no funcionales específicos.

¿Qué es System Design?

En el módulo 01 aprendiste a organizar **código** en capas. System Design opera a otra escala: no diseñas una clase, diseñas **el sistema entero** — la app móvil, el backend, la base de datos, la red, la caché y la infraestructura que los sostiene.

Un sistema está bien diseñado cuando es **reliable (confiable), scalable (escalable) y maintainable (mantenible)**. Estos tres atributos son el marco que usa *Designing Data-Intensive Applications* (Kleppmann) para evaluar cualquier sistema de datos.



Los 3 atributos clave (DDIA)

Fuente: Kleppmann, *Designing Data-Intensive Applications* (O'Reilly).

1. Reliability (Confiabilidad)

"El sistema sigue funcionando correctamente incluso cuando algo sale mal", aunque falle hardware, software o el humano que opera. Eso implica:

- Tolerancia a fallos (fault tolerance): un servidor cae → no se cae todo.
- Manejo de errores del usuario y del sistema.

- Monitoreo para detectar problemas antes de que afecten a los usuarios.

2. Scalability (Escalabilidad)

"El sistema puede manejar crecimiento de carga sin degradarse". La carga se describe con **parámetros de carga** (load parameters):

- Requests por segundo (RPS/QPS).
- Ratio de lecturas vs escrituras.
- Usuarios activos concurrentes.
- Tamaño de los datos.

Si el sistema es lento para **un solo usuario**, es un problema de *performance*. Si es rápido para uno pero lento con mucha carga, es un problema de *scalability*. (*System Design Primer* — "*Performance vs scalability*".)

3. Maintainability (Mantenibilidad)

"Distintos tipos de personas (ingenieros, ops) pueden trabajar en el sistema de forma productiva". Se logra con:

- **Operabilidad:** logs, métricas, debugging fáciles (módulo 19 del repo).
- **Simplicidad:** reducir complejidad accidental.
- **Evolvabilidad:** poder cambiar y añadir features sin romper lo existente (módulo 01).

System Design vs Clean Architecture vs Diseño de Features

	Clean Architecture (M01)	Diseño de Features (M02)	System Design (M22)
Pregunta	¿Cómo organizo el código?	¿Qué construyo y cómo encaja?	¿Cómo funciona el sistema completo?
Escala	1 archivo / 1 clase	1 feature	Todo el ecosistema
Artefacto	Carpetas y clases	Documento de diseño (FADER)	Diagrama de componentes + trade-offs
¿Toca código?	Sí	Solo diseño	Diagramas y decisiones de infraestructura
Nº de "computadoras"	1 (el dispositivo)	1 app + 1 backend	Muchas (servidores, CDN, replicas)

Son **capas complementarias**. Puedes tener Clean Architecture impecable y que tu sistema caiga con 100k usuarios por no diseñar el backend. Y a la inversa: una infraestructura perfecta no salva código espagueti.

Los 4 pasos universales de diseño

Fuente: *System Design Primer* (donnemartin) y *Grokking Modern System Design Interview* (Educative).

1. CLARIFICAR

→ Definir casos de uso, restricciones y suposiciones.
(¿Quién lo usa? ¿Cuántos usuarios? ¿Ratio lectura/escritura?)
2. DISEÑAR

→ Diseño de alto nivel: componentes principales y conexiones.
3. PROFUNDIZAR

→ Diseño de componentes núcleo: esquema, API, caché.
4. ESCALAR

→ Identificar cuellos de botella y aplicar patrones (LB, cache, sharding).

Regla de oro: *"Everything is a trade-off"* — cada decisión (caché vs consistencia, SQL vs NoSQL, consistencia vs disponibilidad) tiene pros y contras que debes poder justificar.

Errores comunes de principiante

Error	Solución
Empezar a diseñar sin clarificar requisitos	Siempre preguntar primero (usuarios, carga, latencia)
Solo hablar de código	Hablar de componentes: cliente, API, DB, cache, CDN
Ignorar el trade-off	Nombrar explícitamente qué ganas y qué sacrificas
No mencionar escalabilidad	Incluir siempre el paso "escalar"
Inventar números de latencia	Usar la tabla real de latencias (ver archivo 02)

Fuentes

- Designing Data-Intensive Applications — M. Kleppmann
- The System Design Primer — donnemartin
- Grokking Modern System Design Interview — Educative

Siguiente: 01-clarificar-requisitos.md

01: Clarificar Requisitos

El paso más infravalorado del system design. Una entrevista o un proyecto se ganan o se pierden en los primeros 10 minutos, cuando **claras el problema antes de diseñar**.

Por qué clarificar primero

El system design interview es una **conversación abierta**. Se espera que *tú* la lideres. Si empiezas a dibujar servidores sin saber cuántos usuarios hay, estás diseñando a ciegas.

Fuente: System Design Primer — "How to approach a system design interview question".

Step 1: Outline use cases, constraints, and assumptions. Gather requirements and scope the problem. Ask questions to clarify use cases and constraints.

Framework de clarificación

A. Casos de uso (¿qué hace el sistema?)

- ¿Quién lo usa? (usuarios, admins, sistemas externos)
- ¿Cómo lo usan? (app móvil, web, ambos)
- ¿Cuáles son las entradas y salidas del sistema?
- ¿Cuál es el caso de uso principal y cuáles son secundarios?

B. Restricciones (¿cuánto, qué tan rápido, cuántos?)

Pregunta	Determina
¿Cuántos usuarios totales?	Complejidad del sistema
¿Cuántos usuarios concurrentes?	Capacidad de servidores
¿Cuántos requests por segundo esperamos?	Dimensionamiento
¿Cuánta data por día?	Almacenamiento
¿Ratio de lecturas vs escrituras?	Patrón de cache/replicación

Pregunta	Determina
¿Latencia aceptable?	Caching, CDN, geografía
¿Requiere real-time?	WebSocket vs polling
¿Disponibilidad requerida?	Nº de "nueves", failover

C. Suposiciones (lo que asumes y declaras)

Todo lo que no te digan, lo **asumes en voz alta** y lo dejas escrito. Ejemplo: "Asumo 100M de lecturas/día y 10M de escrituras/día". Esto convierte el problema abierto en algo accionable.

Ejemplo aplicado al stack del repo: "Diseña un feed de posts"

Aplicando el framework a una feature Flutter + Supabase:

CASOS DE USO

- Usuario autenticado ve un feed paginado de posts (principal)
- Usuario crea un post con imagen (secundario)
- Usuario da like (secundario)

RESTRICCIONES (las preguntamos)

- ¿Cuántos usuarios? → 500k registrados, 50k DAU
- QPS de lecturas? → $50k \text{ DAU} \times \sim 10 \text{ lecturas/día} / 86400s \approx 6 \text{ QPS promedio}$, picos de 30 QPS
- Ratio L/E? → ~ 10 lecturas por cada escritura
- Latencia? → < 500 ms al hacer scroll
- Realtime? → No imprescindible, se acepta refresh

SUPOSICIONES DECLARADAS

- Las imágenes se sirven vía CDN
- Los posts viven en Postgres (Supabase)
- Asumo que el pico es 5× el promedio

Este párrafo de 6 líneas **define todo el diseño posterior**: si el ratio es 10:1, priorizamos cache de lecturas; si es realtime, Supabase Realtime.

Técnica del "si no me lo dicen, lo asumo"

En una entrevista el entrevistador suele ser deliberadamente vago. La técnica correcta:

NO: "Diseño un feed" (y empiezas a dibujar)

SÍ: "Antes de diseñar necesito saber: ¿cuántos usuarios?, ¿cuántos requests por segundo?, ¿el feed es en tiempo real o con refresh?, ¿las imágenes se sirven por CDN? Si no me lo puedes decir, asumo 50k DAU y ratio 10:1."

Esto demuestra criterio y cubre tus espaldas: si asumiste mal, es responsabilidad documentada.

Checklist de clarificación

- ☐ Usuarios totales y activos (DAU/MAU)
 - ☐ Requests por segundo (promedio y pico)
 - ☐ Ratio lectura/escritura
 - ☐ Almacenamiento esperado (GB/TB por día)
 - ☐ Latencia objetivo
 - ☐ ¿Realtime o no?
 - ☐ Disponibilidad objetivo (SLA)
 - ☐ Plataformas (móvil, web, ambos)
 - ☐ Suposiciones declaradas en voz alta
 - ☐ Escenario de crecimiento (¿×10 en 1 año?)
-

Fuentes

- [The System Design Primer — How to approach a system design interview question](#)
 - [Grokking Modern System Design Interview — Educative](#)
-

Siguiente: [02-estimacion-de-escala.md](#)

02: Estimación de Escala (Back-of-the-Envelope)

Traducir usuarios en **requests por segundo, almacenamiento y ancho de banda**. Se llama "back-of-the-envelope" porque se hace a mano, en el dorso de un sobre, con números redondos.

¿Por qué estimar?

Estimar antes de diseñar te dice:

- Cuántos servidores/instancias necesitas (coste).
- Si una sola base de datos aguanta o necesitas replicación/sharding.
- Si necesitas CDN para imágenes/videos.
- Si el diseño es viable en la nube elegida (Supabase, AWS, etc.).

Fuente: *System Design Primer* — "Back-of-the-envelope calculations".

Tablas de referencia REALES (no inventadas)

Estas son las tablas que cita el System Design Primer. **No las improvises** en una entrevista: memorízalas.

Potencias de 2 (referencia rápida)

Abreviatura	Bytes	Valor
KB	2^{10}	1,024
MB	2^{20}	~1 millón
GB	2^{30}	~1 billón
TB	2^{40}	~1 trillón

Números de latencia que todo programador debería saber (2020)

Operación	Tiempo
Lectura de caché en memoria	~1 ns
Compresión/decompresión de 1 KB	~1-2 μ s

Operación	Tiempo
Lectura secuencial de 1 MB en RAM	~9 μ s
Round-trip de datacenter (misma región)	~500 μ s
Búsqueda de 1 MB secuencial en SSD	~150 μ s
Disco rotativo (seek + lectura 1 MB)	~2-4 ms
Round-trip en Internet (EE.UU. → Europa)	~150 ms
Round-trip en Internet (EE.UU. → Asia)	~200 ms

⚠ Estos valores los publica el System Design Primer y la comunidad los actualiza periódicamente. Al citarlos, referencia la fuente; son órdenes de magnitud, no medidas exactas.

Método de 4 pasos

1. Encuentra los números base → usuarios, uso diario
2. Calcula requests/segundo → QPS promedio y pico
3. Calcula almacenamiento → bytes por día y por año
4. Calcula ancho de banda → bytes por segundo

Ejemplo completo: feed de 500k usuarios

Datos de entrada (de la clarificación del archivo 01):

Usuarios registrados : 500,000
 Usuarios activos : 50,000 DAU
 Uso por usuario/día : 10 lecturas (scroll de feed) + 1 escritura (post)
 Tamaño de un post : 1 KB de texto + 500 KB de imagen (media)

Paso 2: QPS

Lecturas/día = $50,000 \times 10 = 500,000$ lecturas
 Escrituras/día = $50,000 \times 1 = 50,000$ escrituras
 Total/día = 550,000

Segundos en un día = 86,400

QPS promedio = $550,000 / 86,400 \approx 6.4$ QPS
 QPS pico = 6.4×5 (factor de pico) ≈ 32 QPS

Interpretación: ~32 QPS pico es un número pequeño. Una sola instancia bien configurada lo soporta. La complejidad real vendrá de la **data acumulada** y de **medios**, no del volumen de requests.

Paso 3: Almacenamiento

Posts por día = 50,000

Texto/día = 50,000 × 1 KB = 50 MB/día

Imágenes/día = 50,000 × 500 KB = 25 GB/día

Total/día ≈ 25 GB/día (dominado por las imágenes)

Total/año ≈ 25 GB × 365 ≈ 9.1 TB/año

Interpretación: 9 TB/año es mucho. Las imágenes **no deben vivir en la base de datos**: van a un object storage (Supabase Storage) servido por CDN. En Postgres solo guardamos la URL.

Paso 4: Ancho de banda

QPS promedio = 6.4

Bytes/request ≈ 500 KB (una imagen por request)

Entrada de escrituras ≈ 50,000 × 500 KB / 86,400 s ≈ 290 KB/s

Salida de lecturas ≈ 500,000 × 500 KB / 86,400 s ≈ 2.9 MB/s promedio

Salida en pico (32 QPS) ≈ 32 × 500 KB ≈ 16 MB/s

Interpretación: la salida de 2.9 MB/s promedio (16 MB/s pico) justifica un CDN: los usuarios reciben las imágenes desde el edge más cercano y el backend deja de gastar ese ancho de banda.

Conclusión del diseño (qué nos dice la estimación)

Hallazgo	Decisión de diseño
QPS pico bajo (~32)	Una instancia de app basta en el inicio
9 TB/año de imágenes	Imágenes → Storage + CDN, nunca en la DB
Ratio lectura/escritura 10:1	Cachear el feed de lecturas
Picos 5× promedio	Margen de capacidad y auto-scaling

Este es el ciclo completo: **clarificar** → **estimar** → **diseñar informado**.

Errores comunes

Error	Solución
Calcular QPS sin aclarar usuarios	Primero clarifica, luego estima
Guardar imágenes en la DB	Separar media (Storage/CDN) de metadatos (DB)
Redondear mal los órdenes de magnitud	Usar las potencias de 2 de referencia
Confundir MB con MB decimales	Ser consistente: usar $2^{20} = 1,048,576$ bytes
No declarar suposiciones	Escribir los números base antes de operar

Fuentes

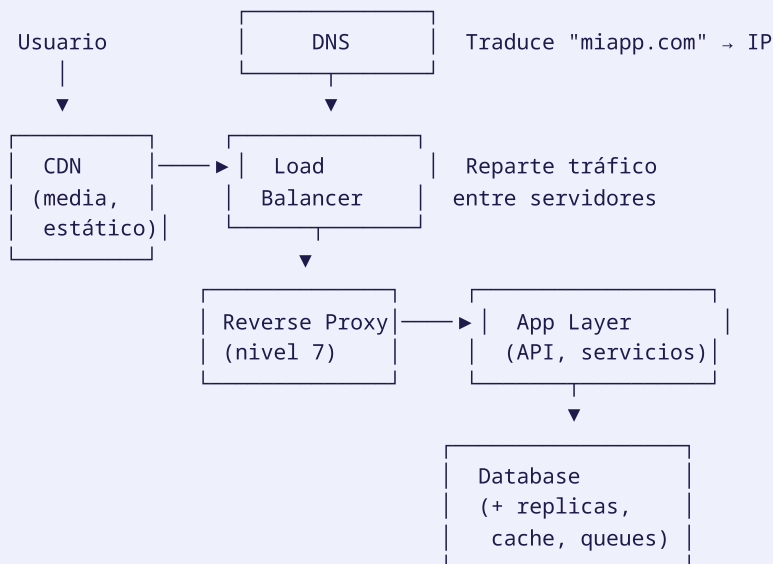
- [The System Design Primer — Back-of-the-envelope calculations](#)
- [The System Design Primer — Powers of two table](#)
- [Grokking Modern System Design Interview — Back-of-the-Envelope Calculations](#)

Siguiente: [03-componentes-arquitectura.md](#)

03: Componentes de Arquitectura

Los bloques con los que se construye todo sistema escalable. Aprender a *nombrar* cada componente y *justificar* su presencia (o ausencia) es la mitad de un system design.

El mapa completo



Componentes tomados del índice de topics del System Design Primer.

1. DNS (Domain Name System)

Traduce un nombre (**miapp.com**) a una dirección IP. Es jerárquico y tiene caching en varios niveles (browser, OS, servidores).

Tipos de registros:

- **A record** — nombre → IP.
- **CNAME** — nombre → otro nombre.
- **MX** — servidores de correo.

En el diseño:

- El DNS introduce un pequeño delay, mitigado por el TTL (time to live) del caching.
- Servicios gestionados: Cloudflare, Route 53. Pueden enrutar por **latency** o **geolocalización** (para servir la región correcta).

2. CDN (Content Delivery Network)

Red global de proxies que sirve contenido desde el nodo **más cercano al usuario**. Ideal para estáticos: HTML/CSS/JS, fotos, videos.

Push vs Pull CDN

	Push CDN	Pull CDN
Quién sube el contenido	Tú, cuando cambia	El CDN, en la primera request
Uso típico	Poco tráfico, contenido que rara vez cambia	Tráfico alto, contenido reciente
Trade-off	Maximiza almacenamiento	Minimiza almacenamiento, puede re-pull innecesario

En un stack Flutter + Supabase

- Las **imágenes de los posts** van a **Supabase Storage** con CDN/edge de entrega, no a la base de datos (lo calculamos en el archivo 02: 9 TB/año).
- La app apunta a URLs de media, no a endpoints de la API, para ese tráfico.

Fuente: *System Design Primer* — "Content delivery network".

3. Load Balancer

Distribuye requests entrantes entre varios servidores. Sus beneficios:

- Evita enviar tráfico a servidores no saludables (health checks).
- Evita sobrecargar un recurso.
- Elimina puntos únicos de fallo.
- **SSL termination**: descifra HTTPS para que el backend no lo haga.

Nivel 4 vs Nivel 7

	Layer 4 (transport)	Layer 7 (application)
Decide por	IP origen, IP destino, puerto	Header, mensaje, cookies

	Layer 4 (transport)	Layer 7 (application)
Tráfico	Adelanta paquetes (NAT)	Termina la conexión, lee, decide, reconecta
Ejemplo	LB de red	Nginx, HAProxy (routing por URL)

Algoritmos de reparto

- Round robin / weighted round robin
- Least loaded
- Session/cookies (sticky sessions)

Fuente: *System Design Primer* — "Load balancer".

4. Reverse Proxy (servidor web)

Servidor central que recibe requests y las reenvía a los servidores internos. Diferencia con el LB:

"A load balancer is useful when you have multiple servers. Often, load balancers route traffic to a set of servers serving the same function. A reverse proxy is useful even with just one web server or application server, providing SSL termination, caching, and security benefits."

Beneficios: terminación SSL, caching de respuestas, compresión, mitigación de DDoS.

5. Application Layer (capa de aplicación)

Donde vive la lógica de negocio. Aquí entra la decisión: **monolito vs microservicios**.

Microservicios

- Descomponen la app en servicios independientes, cada uno con su propia data.
- Añaden complejidad: **service discovery**, comunicación entre servicios, datos distribuidos.
- **Regla práctica del primer:** si tu app cabe en un monolito (Supabase cubre auth, DB, storage, realtime en un solo servicio), no empieces con microservicios.

En el stack del repo

- **Supabase actúa como "Backend as a Service":** ya te da Auth, Postgres, Storage, Realtime y Edge Functions como componentes del "application layer" gestionado.
- Las **Edge Functions (Deno)** son el lugar para lógica que no debe vivir en el cliente: webhooks, procesamiento, notificaciones.

Fuente: *System Design Primer* — "Application layer".

6. Service Discovery

Cuando hay muchos servicios, un cliente necesita saber **qué instancia** de qué servicio contactar.
Mecanismos:

- **DNS-based:** el cliente consulta un registro y recibe una IP.
- **Service registry:** un registro central (Consul, etcd, ZooKeeper) que guarda instancias vivas.

En Supabase esto es transparente: el cliente habla con el endpoint gestionado y Supabase balancea internamente.

7. Communication (cómo se comunican)

Protocolo	Uso típico	Notas
REST	APIs CRUD	Stateless, HTTP, es la base de Supabase REST
RPC	Llamadas a funciones	Supabase RPC (Postgres functions)
WebSocket	Realtime bidireccional	Ver archivo 06
TCP/UDP	Capa transporte	UDP para baja latencia (voz, video)

Cómo decidir qué componentes usar

Pregunta guía por componente:

Componente	¿Cuándo lo necesito?
CDN	Imágenes/videos o usuarios geográficamente dispersos
Load Balancer	2+ instancias de un mismo servicio
Reverse Proxy	Un solo servidor y necesitas SSL/cache/compresión
Cache (Redis)	Lecturas repetitivas de los mismos datos
Message queue	Trabajo pesado que puede esperar (async)
Replicas de DB	Carga de lectura alta (ratio L/E alto)

No todos los componentes en el diagrama son obligatorios. Un MVP Flutter + Supabase no necesita un LB propio: Supabase ya lo maneja. El arte está en **justificar cada pieza**.

Fuentes

- [The System Design Primer — Index of system design topics](#)
 - [The System Design Primer — DNS](#)
 - [The System Design Primer — CDN](#)
 - [The System Design Primer — Load balancer / Reverse proxy](#)
 - [The System Design Primer — Application layer](#)
 - [ByteByteGo — Reverse Proxy vs. API Gateway vs. Load Balancer](#)
 - [Supabase — Features](#)
-

Siguiente: [04-modelado-de-datos.md](#)

Diseñar el esquema no solo para que funcione hoy, sino para que sobreviva a millones de filas. Índices, replicación, sharding, denormalización y la decisión SQL vs NoSQL.

```

1 fila → 1M filas → 100M filas → 1B filas
|           |           |           |
|           Índices     Replicas     Sharding
|           + cache     (lecturas)   (escrituras)
|
(aquí vive la mayoría de MVPs)

```

```
-- Posts con índice por usuario y por fecha
CREATE TABLE posts (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  image_url TEXT NOT NULL,
  caption TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_posts_user ON posts(user_id);
CREATE INDEX idx_posts_created ON posts(created_at DESC);
```

Regla: el índice se crea para las **consultas que realmente haces**, no para todas las columnas. Cada índice acelera lecturas pero frena escrituras.

2. La decisión SQL vs NoSQL

Criterio	SQL (PostgreSQL)	NoSQL (documento/llave-valor)
Esquema	Fijo, relacional	Flexible
Transacciones ACID	Sólidas	Variables según motor
Relaciones/joins	Nativos	Se emulan en app
Escalado	Vertical + réplicas; sharding manual	Horizontal por diseño
Ideal para	Datos relacionados, transacciones	Flexibilidad, escrituras masivas

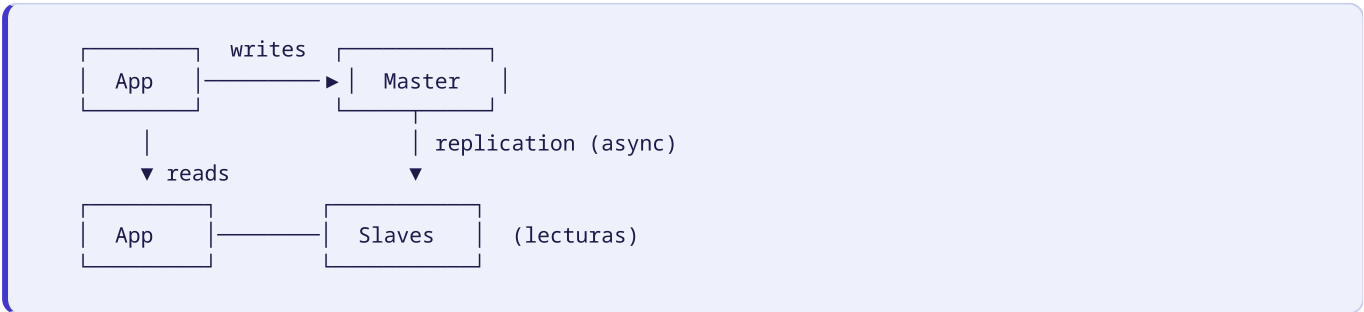
Supabase usa PostgreSQL. Para el 90% de las apps Flutter + Supabase, la respuesta es SQL. Los casos NoSQL suelen aparecer a escalas enormes o con datos no estructurados.

Fuente: *System Design Primer* — "SQL or NoSQL".

3. Replicación

Copias de la misma base de datos para alta disponibilidad y/o lecturas distribuidas.

Master-Slave (activo-pasivo)



- El master atiende escrituras y se replica a los slaves.
- Los slaves atienden lecturas → más throughput de lectura.
- **Trade-off:** si el master muere antes de replicar una escritura, esa escritura se pierde.

Master-Master (activo-activo)

- Ambos atienden escrituras; se replican entre sí.
- **Trade-off:** resolución de conflictos entre dos masters; complejidad alta.

En Supabase/Postgres: Supabase ofrece réplicas de lectura (read replicas) como función de pago. El diseño se documenta igual; el aprovisionamiento lo gestiona Supabase.

4. Federación (Federation)

Dividir las bases de datos **por dominio/función** en vez de por datos:

Una sola DB —► DB de usuarios + DB de posts + DB de analytics

Pros: menos contención, aislamiento de fallos. **Contras:** joins entre DBs se vuelven costosos; más infraestructura.

En el diseño Flutter+Supabase: equivale a separar tablas en esquemas y, a escala mayor, servicios independientes con su propia DB. Para un MVP no se recomienda.

5. Sharding (particionado horizontal)

Distribuir filas de una tabla en **múltiples bases de datos** según una clave (shard key):

Hash(user_id) % 3

Shard 0	Shard 1	Shard 2
1/3	1/3	1/3

Trade-offs del sharding:

- **Pros:** escala escrituras y almacenamiento horizontalmente.
- **Contras:** joins entre shards, operaciones no distribuidas en app, rebalanceo complejo, agregados difíciles.

Regla: sharding es el **último recurso**, después de índices, cache, réplicas y federación. En Supabase, esto se sale del alcance gestionado; a esa escala ya no es una sola instancia de Postgres.

6. Denormalización

Guardar datos **redundantes** a propósito para evitar joins costosos en las lecturas.

Normalizado: posts + join a users para mostrar el nombre del autor

Denormalizado: posts.autor_name ya guardado en la tabla de posts

Trade-off: lees más rápido, pero **escribes más** (mantener la copia sincronizada) y arriesgas inconsistencias.

Ejemplo real del repo (feed): en vez de un join `posts ⋈ users ⋈ likes` en cada scroll, el feed cacheado se guarda **ya ensamblado** (denormalizado) con nombre de autor e imagen. El join se paga una vez al ensamblar el cache, no en cada lectura.

7. SQL tuning (ajuste de consultas)

Antes de sharding, primero se optimiza la consulta:

- Usar **índices** adecuados (composite indexes para filtros múltiples).
- **EXPLAIN** para ver el plan de ejecución (en Supabase: `EXPLAIN ANALYZE`).
- Evitar `SELECT *`, limitar columnas.
- Paginación con **cursor** en vez de `OFFSET` grande para feeds.

Árbol de decisión de escalado de datos

¿La consulta es lenta?

— Sí → ¿Índices? → NO → añade índices + EXPLAIN

— Sí → ¿Es por lectura repetitiva? → cache (archivo 05)

— NO → ¿Muchas lecturas concurrentes? → read replicas

— ¿Escrituras el cuello? → federación / particionado

— ¿Todo lo anterior falla? → sharding

— No → sigue así

En el stack Supabase: qué es de tu diseño y qué es gestionado

Pieza	¿La diseñas tú?	¿La gestiona Supabase?
Esquema, índices, RLS	✅ Sí (módulo 03 del repo)	No
Replicación	Documentas el diseño	✅ Read replicas (función)
Alta disponibilidad/failover	No	✅ Gestionado
Backups	No	✅ Gestionado
Sharding	Solo a escala gigante	No (fuera del alcance)

Fuentes

- The System Design Primer — Database
- The System Design Primer — SQL or NoSQL
- Designing Data-Intensive Applications — M. Kleppmann
- Supabase Docs — Database

Siguiente: [05-cache-y-rendimiento.md](#)

05: Cache y Rendimiento

La caché es la herramienta de rendimiento #1 en cualquier sistema: evita rehacer trabajo caro (lecturas a la DB, redes, cálculos). Aprender las estrategias y sus trade-offs es imprescindible.

Latencia vs Throughput

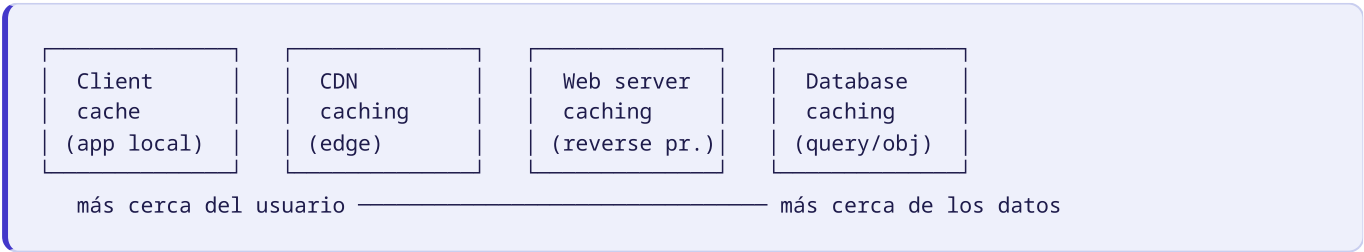
Antes de cachear, hay que medir bien:

- **Latency:** tiempo en realizar una acción o producir un resultado.
- **Throughput:** número de acciones o resultados por unidad de tiempo.
- **Objetivo general:** "maximal throughput with acceptable latency".

Fuente: System Design Primer — "Latency vs throughput".

Un problema de latencia = tu sistema es lento para un usuario. Un problema de throughput = es rápido para uno pero se hunde con muchos.

Dónde poner cachés (de cliente a servidor)



Fuente: System Design Primer — "Cache" section.

Capa	Qué cachea	Ejemplo en Flutter + Supabase
Cliente	Datos ya vistos, respuestas	Cache local del feed (Isar, módulo 04 del repo)
CDN	Estáticos/media	Imágenes de Supabase Storage
Servidor web	Respuestas HTTP completas	Reverse proxy, respuestas de API
Aplicación	Objetos serializados	Cache en memoria/Redis de respuestas
Base de datos	Resultados de queries	Query cache, materialized views

Niveles de caché en detalle

Client caching (en la app Flutter)

- **HTTP caching:** headers `Cache-Control`, `ETag` para no re-descargar lo que no cambió.
- **Local DB:** guardar en Isar las últimas N páginas del feed → lecturas offline y pantalla instantánea.
- **Beneficio enorme:** evita round-trips completos; es la caché más barata.

CDN caching

- Sirve media desde el nodo más cercano (ver archivo 03).
- TTL controla cuánto tiempo el contenido se considera válido.

Web server / application caching

- Una caché en memoria (p. ej. Redis o un cache del servidor) guarda las **respuestas frecuentes ya computadas**.
- Para Supabase: las lecturas del feed se pueden cachear en una Edge Function o en un servicio intermedio.

Database caching

- Resultados de queries frecuentes en memoria (query cache).
- **Materialized views** en Postgres para agregados precomputados (ej. conteos de likes).

Estrategias de actualización de caché

Fuente: *System Design Primer* — "When to update the cache".

1. Cache-aside (lazy loading)

```
App → busca en cache → ¿hit? → devuelve
                        → ¿miss? → lee de DB → guarda en cache → devuelve
```

- **Pros:** datos solo se cachean si se usan; tolerante a fallos de cache.
- **Contras:** primer acceso lento (miss); si el cache muere, la DB recibe todo.

2. Write-through

- La escritura va a la caché **y** a la DB en el mismo paso.
- **Pros:** consistencia entre cache y DB; lecturas nunca con datos viejos.
- **Contras:** cada escritura paga doble (mayor latencia de escritura); riesgo de datos poco usados ocupando cache.

3. Write-behind (write-back)

- La escritura va solo a la caché y se persiste a la DB **asíncronamente**.
- **Pros:** escrituras rapidísimas.
- **Contras:** riesgo de perder datos si la caché cae antes de persistir.

4. Refresh-ahead

- La caché se refresca **antes** de que expire si predice que será accedida.
- **Pros:** menos misses.
- **Contras:** predicción puede equivocarse (work innecesario).

Estrategia	Velocidad de lectura	Consistencia	Riesgo
Cache-aside	Miss lento	Stale hasta refresco	Cache miss storm
Write-through	Siempre fresca	Alta	Escrituras lentas
Write-behind	Alta	Eventual	Pérdida de datos
Refresh-ahead	Alta	Eventual	Work innecesario

Consideraciones de caché (trade-offs)

- **TTL (expiración):** el contenido puede quedar **stale** (desactualizado) hasta que expire.
- **Invalidación:** cuando un dato cambia, hay que invalidar/actualizar el cache — si no, los usuarios ven datos viejos.
- **Eviction:** políticas para quitar datos cuando el cache se llena (LRU — Least Recently Used — es la más común).
- **Cache stampede (miss storm):** si todos los caches expiran a la vez y miles piden el mismo dato a la DB → desastre. Mitigar con *stale-while-revalidate* o jitter en TTL.

Diseño de caché para el feed del repo (decisión completa)

Datos del archivo 02: ratio lectura/escritura 10:1, QPS pico ~32, feed ensamblado caro.

Diseño elegido (cache-aside + write-through para el feed):

Lectura: app → cache local (Isar, última página) → si miss → API → Redis
→ si miss → Postgres (ensambla con joins) → guarda en Redis → devuelve

Escritura: cuando se crea un post → se invalida el cache del feed del autor
(los feeds de otros se refrescan con cache-aside natural)

Justificación (trade-offs):

- Cache-aside: el feed es mayormente de **lectura** → ratio 10:1 lo justifica.
- Invalidez al escribir: mantener el feed de cada usuario coherente sin write-through costoso.
- Cache local (Isar): pantalla instantánea + soporte offline (enlaza archivo 10).

Errores comunes

Error	Solución
Cachear todo sin medir	Primero medir latencia/throughput
Sin TTL ni invalidación	Definir TTL y ruta de invalidación por escritura
Un solo cache global	Distribuir por capa (cliente, CDN, app, DB)
Ignorar el cache stampede	Jitter en TTL + stale-while-revalidate
Cachear datos que no se leen	Cache-aside cachea solo lo usado

Fuentes

- [The System Design Primer — Cache](#)
- [The System Design Primer — Latency vs throughput](#)
- [ByteByteGo — Cache Systems Every Developer Should Know](#)
- [ByteByteGo — Top 5 Caching Strategies](#)
- [Designing Data-Intensive Applications — M. Kleppmann](#)

Siguiente: 06-realtime-y-streaming.md

06: Realtime y Streaming

Cómo elegir entre polling, WebSocket y Server-Sent Events, y cómo encaja Supabase Realtime. La decisión de realtime cambia el diseño completo del backend.

Las 3 opciones de comunicación en tiempo real

Técnica	Dirección	Cuándo	Ejemplo
Polling	Cliente → servidor (pregunta)	Latencias de segundos, simple	Lista que se refresca cada 30s
Long polling	Cliente → servidor (mantiene abierta hasta respuesta)	Necesito menos polling, sin websocket	Notificaciones simples
WebSocket	Bidireccional, full-duplex	Realtime de verdad, baja latencia	Chat, presencia, live updates
SSE (Server-Sent Events)	Servidor → cliente (unidireccional)	Updates unidireccionales	Feed que se actualiza solo

Fuente: ByteByteGo — "Short/long polling, SSE, WebSocket".

Trade-offs

- **Polling:** simple y funciona en cualquier servidor, pero desperdicia requests (N peticiones por unidad de tiempo aunque no haya datos).
- **WebSocket:** conexión persistente bidireccional, baja latencia; más complejo (manejo de conexiones, reconnection, heartbeats), no funciona tras algunos proxies.
- **SSE:** más simple que WebSocket (HTTP normal, auto-reconnect), pero unidireccional.

Asincronía: la pieza que falta en muchos diseños

No todo tiene que ser síncrono. El System Design Primer lista dos patrones clave:

Message queues

- El productor encola un mensaje; el consumidor lo procesa cuando puede.
- **Desacoplan** componentes: si el consumidor cae, el mensaje espera.

- Ejemplo: "cuando un usuario sube un video, se encola la tarea de transcodificación".

Task queues

- Colas de trabajo pesado (ej. enviar emails, generar thumbnails).
- **Back pressure:** si el consumidor va más lento que el productor, la cola crece — hay que monitorear su tamaño.

En Supabase: las **Edge Functions** + colas/scheduling son el mecanismo para trabajo asíncrono (notificaciones, procesamiento). El trigger clásico del repo: *insert en la DB → trigger → Edge Function → notificación*.

Supabase Realtime (oficial)

Supabase provee Realtime como servicio gestionado. Los tres canales documentados en la documentación oficial:

1. **Postgres Changes:** escuchar inserts/updates/deletes en una tabla y recibir el cambio en la app.
2. **Broadcast:** enviar mensajes arbitrarios entre clientes.
3. **Presence:** saber quién está conectado (útil para chats y presencia).

Arquitectura de Supabase Realtime

Fuente: *Supabase Docs — Realtime architecture*.

```
Postgres —(WAL)→ Realtime —(WebSocket)→ Flutter App
(insert/update)   (detecta cambios)      (recibe en vivo)
```

- La app abre un **WebSocket** a Supabase Realtime.
- El servidor escucha los cambios de Postgres vía el **Write-Ahead Log (WAL)** y los empuja a los clientes suscritos.
- Se puede **filtrar** por evento (INSERT/UPDATE/DELETE), schema, tabla y columnas.

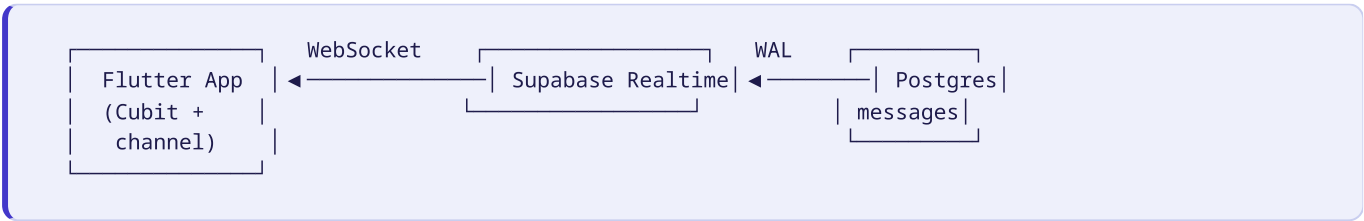
Postgres Changes en la práctica (ejemplo de chat del repo)

```
final channel = supabase
  .channel('messages:room-1')
  .onPostgresChanges(
    event: PostgresChangeEvent.insert,
    schema: 'public',
    table: 'messages',
    callback: (payload) {
      final message = Message.fromJson(payload.newRecord);
      // emitir nuevo estado en el Cubit
    },
  )
  .subscribe();
```

Trade-offs de usar Supabase Realtime

Pros	Contras
No gestionas WebSocket servers	Límites de conexiones por plan
Se integra con RLS (solo ves lo autorizado)	Todo el tráfico pasa por Supabase
Postgres como fuente de verdad	Para broadcast masivo a millones, arquitectura custom

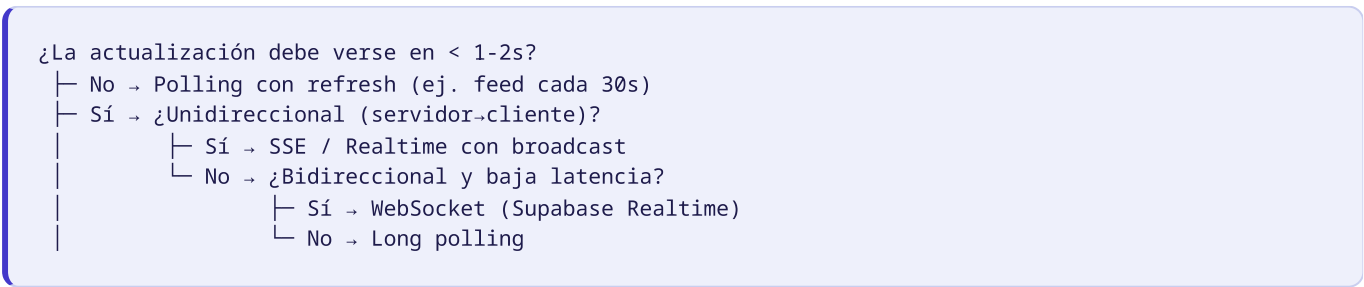
Arquitectura de chat con realtime (resumen de diseño)



1. App suscribe al channel `messages:conversation_id` (con filtro de RLS).
2. Usuario envía → **insert** en Postgres (vía Supabase REST/RPC).
3. Realtime detecta el cambio por WAL y lo empuja a todos los suscritos de la conversación.
4. El Cubit emite el nuevo mensaje → la UI se actualiza sola.

Decisión de diseño a defender: elegimos WebSocket (via Realtime) porque el caso de uso es *chat en vivo* (requiere latencia < segundos y bidireccional). Para un feed no-crítico, polling + refresh sería más barato y suficiente.

Árbol de decisión realtime



Fuentes

- [Supabase Docs — Realtime Architecture](#)

- [Supabase Docs — Realtime Postgres Changes](#)
 - [The System Design Primer — Asynchronism](#)
 - [ByteByteGo — Short/long polling, SSE, WebSocket](#)
-

Siguiente: [07-escalabilidad-alta-disponibilidad.md](#)

07: Escalabilidad y Alta Disponibilidad

Cómo hacer que el sistema soporte más carga y menos caídas. Vertical vs horizontal, CAP theorem, y la disponibilidad medida en "nueves".

Performance vs Scalability

- **Performance:** tu sistema es lento para **un solo usuario**.
- **Scalability:** tu sistema es rápido para uno, pero lento **bajo carga**.

Fuente: *System Design Primer* — "Performance vs scalability".

"A service is scalable if it results in increased performance in a manner proportional to resources added."

Escalado vertical vs horizontal

	Vertical (scale up)	Horizontal (scale out)
Cómo	Más CPU/RAM en la misma máquina	Más máquinas
Límite	Físico (la máquina más grande que exista)	Casi ilimitado
Coste	Caro, con saltos	Lineal, elástico
Fallo	Punto único de fallo	Tolerante (replicas)
Complejidad	Baja	Alta (LB, coordinación)

Regla: primero vertical (barato y simple), después horizontal con **load balancing**.

Fuente: *System Design Primer* — "Horizontal scaling".

CAP theorem

En un sistema distribuido solo puedes garantizar **dos de tres**:

- **Consistency:** cada lectura recibe la escritura más reciente, o un error.

- **Availability:** cada request recibe una respuesta (aunque no sea la más reciente).
- **Partition tolerance:** el sistema sigue operando aunque la red se parta (nodos incomunicados).

"Networks aren't reliable, so you'll need to support partition tolerance. You'll need to make a software tradeoff between consistency and availability."

CP vs AP

	CP (consistencia)	AP (disponibilidad)
Ante una partición	Espera/bloquea hasta estar consistente	Responde con lo disponible (puede estar viejo)
Ideal para	Pagos, inventario, contadores	Feeds, chats, notificaciones
Ejemplo	Postgres replicado con waits	Cache distribuida, DNS

En una app Flutter + Supabase: Postgres por defecto te da consistencia fuerte en una sola instancia. El realtime con cache introduce consistencia eventual donde el negocio lo permita (ej. conteo de likes puede ser eventual; un pago debe ser consistente).

Fuente: System Design Primer — "CAP theorem" + ByteByteGo — CAP Theorem: One of the Most Misunderstood Terms.

Patrones de consistencia

Fuente: System Design Primer — "Consistency patterns".

Patrón	Qué garantiza	Dónde vive
Weak	Tras un write, los reads pueden no verlo (best-effort)	Memcached, VoIP, chat de voz
Eventual	Tras un write, los reads lo verán eventualmente (ms)	DNS, email, feeds
Strong	Tras un write, todos los reads lo ven	Filesystems, RDBMS

En el feed del repo: el contador de likes usa consistencia eventual (así se ve en el ejemplo de 05: cache del feed ensamblado). El estado del pago usa consistencia fuerte (Postgres transaccional).

Patrones de alta disponibilidad

Fuente: System Design Primer — "Availability patterns".

Fail-over

	Active-passive	Active-active
Quién atiende	Solo el activo; el pasivo en standby	Ambos
Heartbeat	El pasivo detecta caída del activo	—
Timeout	Depende de hot/cold standby	—
Coste	Menor	Mayor
Riesgo	Pérdida de writes no replicados	Conflictos entre masters

Replication

Ver archivo 04: master-slave (lecturas distribuidas) y master-master.

Disponibilidad en números (los famosos "nueves")

Disponibilidad	Downtime/año	Downtime/mes	Downtime/día
99%	3d 15h	7h 18m	14m 24s
99.9%	8h 45m	43m 49s	1m 26s
99.99%	52m 35s	4m 23s	8.6s

Tabla publicada por el System Design Primer.

Componentes en serie vs en paralelo

En serie: $Availability_total = A_foo \times A_bar$
 $(0.999 \times 0.999 = 0.998 \rightarrow 99.8\%)$

En paralelo: $Availability_total = 1 - (1 - A_foo) \times (1 - A_bar)$
 $(1 - 0.001 \times 0.001 = 0.999999 \rightarrow 99.9999\%)$

Lectura: cada componente en serie *reduce* la disponibilidad total; los componentes redundantes en paralelo la *aumentan*. Por eso la alta disponibilidad se logra con redundancia (paralelo), no encadenando piezas únicas.

Qué significa esto para Flutter + Supabase

Pieza	¿De quién es la disponibilidad?	Nivel típico
App en el dispositivo	Del usuario (fuera de tu control)	—
Supabase (API, Auth, DB, Storage, Realtime)	De Supabase (SLA gestionado)	Alta

Pieza	¿De quién es la disponibilidad?	Nivel típico
CDN de media	Del proveedor de CDN	Muy alta
Edge Functions	De Supabase	Alta

Diseño que debes defender: no reinventas la alta disponibilidad para Supabase (es gestionada). Tu responsabilidad es:

- 1. **No crear puntos únicos de fallo en tu código** (ej. un `Stream` que muere sin reconnect).
- 2. **Elegir el SLA correcto** para tu caso (un MVP no necesita 99.99%).
- 3. **Cache local** para que la app no caiga si la red falla (offline-first, archivo 10).

Árbol de decisión de escalabilidad

¿Lento con mucha carga?

- └ ¿CPU/RAM saturada? → vertical primero
- └ ¿Lecturas? → cache + read replicas
- └ ¿Escrituras? → federación / particionado
- └ ¿Un servidor único? → LB + horizontal scaling
- └ ¿Caídas? → failover + replicación
- └ ¿Todo a la vez? → rediseña (CAP trade-off explícito)

Fuentes

- The System Design Primer — Performance vs scalability
- The System Design Primer — Availability vs consistency (CAP)
- The System Design Primer — Availability patterns
- ByteByteGo — CAP Theorem: One of the Most Misunderstood Terms
- ByteByteGo — How to Design for High Availability
- Designing Data-Intensive Applications — M. Kleppmann

Siguiente: 08-seguridad-en-diseno.md

08: Seguridad en el Diseño

La seguridad se decide en el diseño, no al final. Esquema de cifrado, authn vs authz, HTTPS, TLS y el principio de menor privilegio — siempre con las herramientas que Supabase ya te da.

Seguridad en capas (defense in depth)

App (Flutter)
· Secrets nunca en el bundle
Transporte
· HTTPS / TLS obligatorio
· Certificados válidos
API (Supabase)
· Auth (identidad)
· RLS (autorización por fila)
Base de datos (Postgres)
· Menor privilegio
· Backups cifrados

HTTPS (cifrado en tránsito)

- **HTTPS** cifra el tráfico entre la app y el servidor. *"HTTPS is the backbone of any secure system."* — System Design Primer.
- Protege de **man-in-the-middle** (alguien leyendo/modificando lo que viaja).
- En Supabase, todas las URLs que usas desde la app son `https://` — el cifrado en tránsito ya está gestionado.

Configuración que sí te toca: nunca apuntar la app a URLs `http://` en producción, y validar el certificado (no desactivar la verificación, ni siquiera "para probar").

Cifrado en reposo (at-rest)

Capa	Qué cifrar	Cómo
En la app	Datos sensibles locales (tokens)	<code>flutter_secure_storage</code> (Keychain/Keystore)
Transporte	Todos los requests	TLS vía HTTPS de Supabase
Base de datos	Datos en el servidor	Cifrado gestionado por Supabase + columnas sensibles extra
Secreto del app	—	Nunca en el bundle; usar Env vars / <code>.env</code> y no commitear

Autenticación vs Autorización (authn vs authz)

- **Authentication (authn):** *"¿quién eres"*. Probar identidad (login, OAuth, email magic link).
- **Authorization (authz):** *"¿qué puedes hacer"*. Decidir permisos sobre esa identidad.

Error típico: validar solo authn (que el usuario esté logueado) y olvidar authz (que ese usuario en concreto tenga permiso sobre **esa fila**).

Rol del JWT en Supabase

- Al hacer login, Supabase emite un **JWT** (JSON Web Token) con el `sub` (user id) y los claims de rol.
- El JWT se envía en el header `Authorization: Bearer <token>` en cada request.
- Los **RLS policies** en Postgres leen ese token (vía `auth.uid()`) para decidir qué filas puede ver/modificar cada usuario.

Row Level Security (RLS) como diseño de autorización

El módulo 03 del repo ya cubre RLS en detalle. Aquí la **visión de diseñador**:

- **RLS es tu authorization layer principal** en Supabase: se evalúa en la DB, no en la app. Aunque alguien llame a la API directamente con un JWT robado, RLS sigue protegiendo.
- Diseño por defecto: **deny by default**, luego políticas explícitas de lectura/escritura.
- Cada política usa `auth.uid()` (el `sub` del JWT) → la autorización queda ligada a la identidad autenticada.

```
-- Ejemplo: solo el dueño puede ver sus draft posts
CREATE POLICY "read_own_drafts" ON posts
FOR SELECT USING (
  auth.uid() = user_id AND status = 'draft'
);
```

Principio de menor privilegio (least privilege)

"Every program and every user of the system should operate using the least set of privileges necessary to complete the job." — Saltzer & Schroeder.

Aplicado al stack:

Quién	Permisos correctos
Anon key en la app	Solo lo público (RLS niega el resto)
Service role key	Nunca en la app; solo servidor/Edge Functions
Edge Functions	Solo las operaciones que necesita
DB roles	Solo los schemas/tablas que usan

Principio de defensa en profundidad

- No confiar en **una sola** capa de seguridad (ej. solo login sin RLS, o solo RLS sin HTTPS).
- Cada capa protege si la anterior falla: app segura + TLS + authn + authz + least privilege.

Fuente: System Design Primer — "Security" section + defensa en profundidad clásica (CISSP/OWASP).

Errores de diseño de seguridad comunes

Error	Fix
Service role key en el bundle de Flutter	Mover a Edge Functions / servidor
Confiar solo en authn (login) sin authz	Añadir RLS por fila
Permisos abiertos en Storage buckets	Bucket público vs privado + RLS en storage.objects
Desactivar verificación de certificado TLS	Validar siempre en producción
Loggear tokens/secrets en logs	Nunca imprimir secrets; rotar si se filtra
Cifrar todo sin evaluar riesgo	Cifrar lo sensible (tokens, PII) — el resto no añade valor

Checklist de seguridad de diseño (para tus entrevistas/proyectos)

1.  Todos los requests por HTTPS.

2. ✓ Tokens de Supabase en `flutter_secure_storage`.
 3. ✓ RLS habilitado en todas las tablas de datos del usuario.
 4. ✓ Service role key solo en backend.
 5. ✓ Buckets de Storage con su política (público vs privado).
 6. ✓ No hay secrets en el repo (`.env` en `.gitignore`).
 7. ✓ Principio de menor privilegio en roles y Edge Functions.
 8. ✓ Backups y datos sensibles cifrados en reposo.
-

Fuentes

- [The System Design Primer — Security](#)
 - [Supabase Docs — Row Level Security](#)
 - [Supabase Docs — Storage security \(RLS on buckets\)](#)
 - [Saltzer & Schroeder — The Protection of Information in Computer Systems](#)
 - [OWASP Top 10](#)
-

Siguiente: [09-observabilidad-monitoreo.md](#)

09: Observabilidad, Monitoreo y Resiliencia

Si no puedes medirlo, no puedes arreglarlo. Logs, métricas, tracing y alerts: el sistema debe decirte cuándo y por qué falla — y cómo recuperarse solo.

Observabilidad vs Monitoreo

- **Monitoreo:** saber si un sistema está "sano" (dashboards, checks).
- **Observabilidad:** poder **preguntarle al sistema** qué está pasando ante cualquier condición, incluso las imprevistas.

La observabilidad se logra con **tres pilares**: logs, métricas y tracing.

Los tres pilares

Pilar	Qué es	Pregunta que responde
Logs	Registro de eventos discretos	¿Qué pasó exactamente?
Métricas	Medidas agregadas numéricas	¿Cuánto / con qué frecuencia?
Tracing	Recorrido de un request a través de componentes	¿Dónde se tardó/fracturó?

Logs

- Estructurados (JSON) para poder filtrarlos.
- Niveles: debug / info / warn / error.
- **Cuidado:** logs con PII o secrets = fuga de seguridad (ver archivo 08).

Métricas (con el acrónimo USE)

- **Utilization:** % de recurso usado (CPU, memoria, conexiones de DB).
- **Saturation:** qué tan lleno está (colas, conexiones pool).
- **Errors:** tasa de fallos, excepciones, 5xx.

Tracing

- Un **trace** sigue un request desde la app hasta la DB pasando por Edge Functions.

- Componentes: **spans** (cada tramo), **trace id** (identificador único del request).

Supabase: lo que ya tienes (sin montar infraestructura)

Supabase gestionado expone sus propios logs y métricas (dashboard + API de logs) para:

- **Postgres:** queries lentas, errores, conexiones.
- **Auth:** eventos de login/registro fallidos.
- **Realtime:** conexiones y mensajes.
- **Storage:** requests a los buckets.
- **Edge Functions:** invocaciones, errores, duración.

Fuente: [Supabase Docs — Logging](#) y [Supabase Docs — Metrics](#).

Decisión de diseño: para el MVP usas los logs gestionados de Supabase + analytics de la app. No inventas un stack de observabilidad hasta que el tráfico/errores lo justifiquen.

Analytics y métricas en Flutter (lado cliente)

En la app necesitas saber cómo se comportan **tus usuarios**:

Herramienta	Para qué
Crash reporting	Stacktraces de errores en producción
Analytics de eventos	Qué feature se usa, donde se quedan los usuarios
Custom events	Métricas de negocio (registro completado, post publicado)
Screen view	Navegación y embudos

Alerting: las reglas correctas

Alertar **también es diseño**: alertas mal definidas generan ruido y desensibilizan.

Alertas en 4 pasos

1. **Métrico correcto:** lo que importa al usuario (error rate, latencia p95), no solo del servidor.
2. **Umbral realista:** alertar antes de que sea un incidente, sin alertar por ruido.
3. **Alerta accionable:** siempre con el link al log/trace del incidente.
4. **On-call limitado:** no alertes cosas que nadie puede arreglar.

Ejemplo para el feed del repo

ALERTA "Error rate en GET /api/feed > 5% durante 5 min"

LINK: Logs filtrados por status_code 5xx + trace_id

ACCIÓN: revisar Edge Function / query de feed o saturación de la DB

Resiliencia y self-healing

El sistema debe **recuperarse solo** de fallos transitorios:

Técnica	Qué hace	Dónde aplica
Retries	Reintenta requests fallidos	Network de la app, Edge Functions
Backoff exponencial	Cada retry espera más (evita sobrecargar al servidor caído)	Igual que retries
Timeouts	Una llamada no puede colgarse para siempre	Todos los requests de red
Circuit breaker	Tras N fallos, "abre el circuito" y falla rápido en vez de seguir pegándole al servicio	SDK de red, Edge Functions
Fallbacks	Si la fuente falla, servir otra cosa (cache, datos locales)	App con offline-first

Fuente: [ByteByteGo](#) — *Timeouts, Retries and Backoff*.

El patrón básico

```
request → timeout (ej. 10s)
  → falla → retry con backoff (1s, 2s, 4s...)
  → sigue fallando → circuit breaker abre → falla rápido
  → app usa fallback (cache local) → reporta métrica de error
```

Diseño del plan de observabilidad (para tus casos de estudio)

Cada diseño debe responder:

1. **¿Qué mides?** Error rate, latencia, QPS, saturación de DB.
2. **¿Dónde?** App (crash + analytics) y Supabase (logs gestionados).
3. **¿Qué alertas?** Las 3-5 más importantes con umbral accionable.
4. **¿Cómo se diagnostica?** Cada error debe llevarte del dashboard → log → trace.
5. **¿Cómo se recupera?** Retries + backoff + circuit breaker + fallback offline.

Errores comunes de diseño de observabilidad

Error	Fix
Alertas por todo → ruido	Solo alertas accionables
Solo metrics sin logs/traces	Los 3 pilares juntos
Sin timeouts ni retries	Timeouts siempre; retries con backoff
Logs sin estructura	Logs JSON con trace_id
Sin fallback	Cache/offline como plan B de toda lectura

Fuentes

- [ByteByteGo — Observability \(logs, metrics, tracing\)](#)
- [ByteByteGo — Timeouts, Retries and Backoff](#)
- [Supabase Docs — Logging](#)
- [Supabase Docs — Metrics](#)
- [Brendan Gregg — USE Method](#)

Siguiente: 10-offline-first.md

10: Offline-First y Sincronización

Cómo diseñar la app para que funcione sin red y se sincronice al reconectar. El archivo donde el patrón offline-first (del repositorio) se conecta con el system design.

El problema de diseñar "para la nube"

Las arquitecturas clásicas asumen red disponible. Pero en Flutter móvil **la red no es confiable**: elevadores, metro, avión, batería baja. Si la app muere sin conexión, pierdes usuarios.

Offline-first = la app funciona bien **sin red** como estado por defecto, y la nube es la capa de sincronización.

Los 4 niveles de tolerancia offline

Nivel	Qué guarda	Ejemplo
1. Lectura offline	Caché de lo ya visto	Feed ya cargado
2. Escrituras offline	Acciones en cola local	Like, mensaje, borrador
3. Sync en background	Cola se envía al reconectar	Mensajes enviados después
4. Conflict resolution	Ambos lados editaron lo mismo	Editar mismo post en 2 devices

Estrategias de sincronización (con el sistema en mente)

1. Optimistic UI + cola de escrituras (recomendada para MVP)

1. Usuario da like → se guarda en cola local + UI actualiza al instante
2. Sin red: queda en cola con estado "pending"
3. Al reconectar: se envía a la API en orden
4. Éxito → se marca "synced"; fallo → retry con backoff

2. Estado: local-first, sync as a service

- La DB local (Isar) es la **fuentes de verdad para la UI**.

- La nube (Supabase) es la fuente de verdad **del sistema**.
- La sincronización sincroniza estados, no "la base de datos".

3. Conflicto: quién gana

- **Last-write-wins (LWW)**: gana el último `updated_at` . Simple, pierde trabajo.
- **Merge manual**: ambos cambios se conservan y el usuario decide.
- **Por campo**: el último cambio por cada campo.

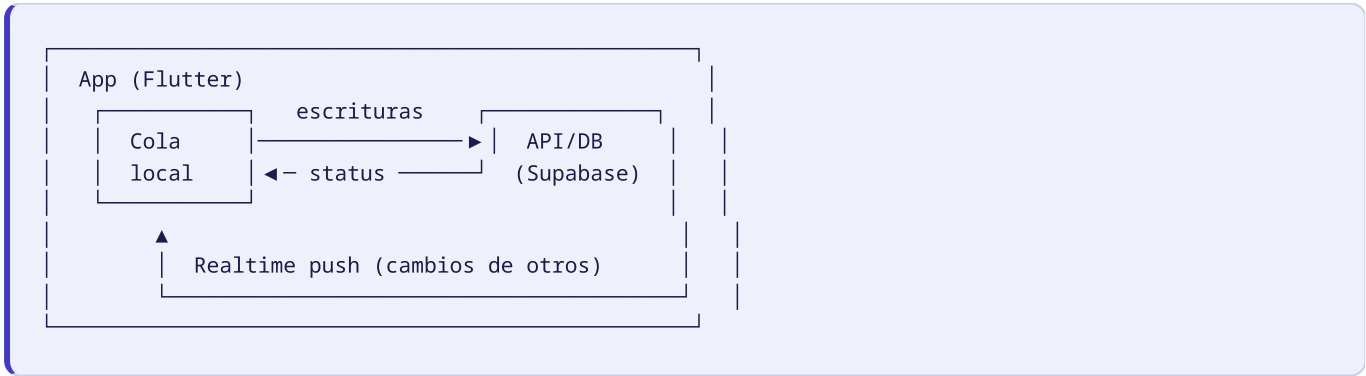
Regla de diseño: define la política de conflictos **antes** de implementar la sincronización. Un conflicto no manejado = datos perdidos silenciosamente.

Sincronización con Supabase

La sincronización del offline-first se apoya en los mecanismos ya vistos:

Mecanismo	Rol en sync
Realtime (archivo 06)	Empujar cambios remotos cuando hay red
RLS (módulo 03)	El servidor decide qué sincronizar según el usuario
UUIDs como PK (módulo 03)	IDs no dependen del orden de creación → sin colisiones de sync
<code>updated_at</code>	Base para LWW y delta sync

El loop de sincronización



La decisión de consistencia (conecta con el archivo 07)

- Con **red disponible**: strong consistency vía Supabase/RLS para datos críticos.
- Con **sin red**: consistencia local inmediata (optimistic) + eventual al sincronizar.

"You can't have both strong consistency and high availability under network partitions." — CAP (archivo 07).

La app offline-first es **AP por diseño**: disponible siempre, consistente eventualmente.

Detección de conectividad y scheduling

- Monitorear conectividad (internet_connection_checker) para saber cuándo flushear la cola.
- **Ejecutar sync al reconectar**, no en loops ciegos.
- Respetar **batería/banda**: no sincronizar media gigante con datos celulares.
- Tareas en segundo plano: ejecutores periódicos en Flutter (workmanager) — mantener simple en MVP.

Errores comunes del diseño offline

Error	Fix
Asumir red siempre disponible	Asumir lo contrario: offline-first
IDs autoincrementales en sync	UUIDs como PK (módulo 03)
Sin política de conflictos	Definir LWW / merge por campo antes
Cola infinita sin backoff	Retries con backoff + límites
Sincronizar todo el tiempo	Sincronizar al reconectar + delta sync
Estado duplicado (UI y sync separados)	Un solo estado (local-first) con cola

Fuentes

- [Supabase Docs — Offline support \(Dart/Flutter\)](#)
- [Redis/DDIA — Conflict resolution](#) (capítulo sobre multileader y conflictos)
- [ByteByteGo — Offline-First Apps](#)
- Patrón del repositorio: módulo 04 (cache local Isar + sync)

Siguiente: 11-plantilla-diseno-sistema.md

11: Plantilla de Diseño de Sistema

El proceso reproducible para cualquier pregunta de system design: del requisito al diagrama, con métricas, decisiones y riesgos. Úsalo en los casos de estudio (12-14) y en tus entrevistas.

La plantilla (10 pasos)

- | | |
|----------------------------------|---|
| 1. Alcance y requerimientos | → qué SÍ y qué NO está en el diseño |
| 2. Estimación de escala | → QPS, storage, ratios (archivo 02) |
| 3. Modelado de datos | → entidades, índices, RLS (archivo 04) |
| 4. Arquitectura de alto nivel | → diagrama de cajas y flujos |
| 5. Componentes clave | → API, DB, cache, CDN, realtime |
| 6. Decisiones de diseño | → cada una con su trade-off |
| 7. Consistencia y disponibilidad | → CAP, SLA, patrón elegido |
| 8. Seguridad | → authn/authz, RLS, HTTPS (archivo 08) |
| 9. Observabilidad | → logs, métricas, alertas (archivo 09) |
| 10. Riesgos y evolución | → cuellos de botella futuros y mitigación |

Paso 1: Alcance y requerimientos

Antes de dibujar, aclara con el entrevistador/cliente:

Pregunta	Ejemplo
¿Quiénes son los usuarios?	Consumidores de un feed de fotos
¿Cuántos?	10M MAU
¿Qué funcionalidad principal?	Ver y publicar posts
¿Tiempo real o eventual?	Feed eventual; chat realtime
¿Mobile, web, ambos?	Flutter (mobile + web)
¿Es un MVP o un sistema maduro?	Determina la complejidad justa

No olvides: definir lo que **NO** está en el alcance (fuera de scope) para no diseñar de más.

Paso 2: Estimación de escala (archivo 02 en 30s)

QPS de lecturas = usuarios activos × acciones por usuario / segundos

QPS de escrituras = publicaciones/transacciones por día / 86,400

Storage = filas × bytes promedio × retención

Ratio lectura/escritura = lecturas / escrituras

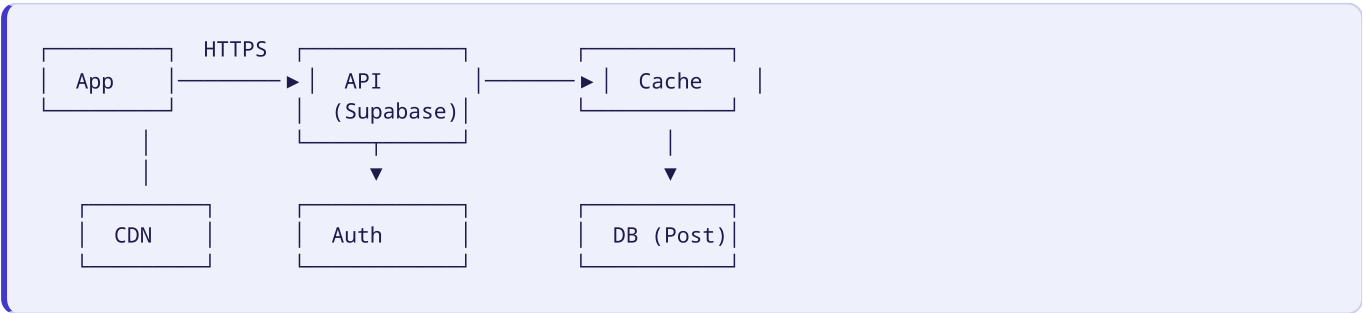
Ejemplo del feed: 10M MAU, 5% activos por hora pico, 100 lecturas diarias → QPS de lectura ≈ 1.7K, escrituras ≈ 4/s, ratio ≈ 10:1.

Paso 3: Modelado de datos (mínimo viable)

- Lista de **entidades** con sus campos clave.
- **Índices** para los path de acceso reales.
- **RLS policies** por entidad (quién lee/escrive).
- Elección: SQL vs NoSQL **justificada** (archivo 04).

Paso 4: Arquitectura de alto nivel

Dibuja las cajas: `App → CDN/API → (Auth, Cache, DB) → Servicios`. Ejemplo genérico:



Paso 5-6: Componentes y decisiones de diseño

Cada componente declarado debe justificarse:

Componente	Decisión	Trade-off
Cache	Cache-aside del feed	Miss lento, pero solo cachea lo usado
Realtime	WebSocket (Realtime)	Complejidad, límites de conexiones
CDN	Para media	Coste, TTL de invalidación

Componente	Decisión	Trade-off
DB	Postgres (Supabase)	Consistencia fuerte, escala limitada

Fórmula: "Elijo X porque el caso es Y, y el trade-off aceptable es Z."

Paso 7: Consistencia y disponibilidad

Declara explícitamente:

- **Modelo de consistencia:** strong (Postgres) para datos críticos; eventual (cache, contadores) para el resto.
- **SLA objetivo:** cuántos "nueves" y por qué.
- **Patrones:** replicas, failover, retries/backoff.

Paso 8-9: Seguridad y observabilidad

- **Seguridad:** HTTPS, RLS deny-by-default, secrets en secure storage, least privilege.
- **Observabilidad:** las 3-5 métricas clave, las alertas accionables, cómo se diagnostica cada error.

Paso 10: Riesgos y evolución

Riesgo detectado	→ Mitigación
-----	→ -----
QPS pico de escrituras	→ Colas + Edge Functions async
Joins caros en el feed	→ Denormalización en cache
Saturación de una DB	→ Replicas de lectura → particionado
App muerta sin red	→ Offline-first (archivo 10)

Cierra la entrevista/diseño con: "el siguiente cuello de botella sería X, y lo resolvería con Y".

Checklist final de una respuesta de system design

- ☐ Alcance claro (incluye fuera de scope)
- ☐ Números justificados (QPS, storage, ratios)
- ☐ Diagrama de alto nivel dibujado
- ☐ Decisiones con trade-offs explícitos
- ☐ Modelo de datos con índices y RLS

- ☐ Consistencia/disponibilidad declaradas
 - ☐ Seguridad y observabilidad presentes
 - ☐ Riesgos futuros y evolución
-

Fuentes

- [The System Design Primer — Intro](#)
 - [ByteByteGo — System Design Interview Guide](#)
 - [System Design Interview — Alex Xu](#)
-

Siguiente: [12-caso-feed-red-social.md](#)

12: Caso de Estudio — Feed de Red Social

Aplicación de la plantilla (archivo 11) al diseño del feed del proyecto del repositorio. Datos del archivo 02, cache del 05 y realtime del 06, unificados en una arquitectura defendible.

Paso 1: Alcance y requerimientos

Producto: feed de fotos estilo Instagram (el feature del repositorio).

	Fuera de scope (este diseño)
Publicar posts	✔ Editar/borrar, stories, reels, monetización, followers graph
Ver feed propio	✔ Notificaciones push, explore page
Likes	✔ Comentarios anidados, DMs

Requerimientos no funcionales: disponibilidad alta para lectura, latencia percibida < 300ms, funciona offline.

Paso 2: Estimación de escala

Del archivo 02 (resumen):

Métrica	Valor
Usuarios	10M MAU, ~500K DAU
QPS de lectura (feed)	~1.7K en pico
QPS de escritura (posts + likes)	~17
Ratio lectura/escritura	~10:1
Storage anual	~2.7 TB de media

Lectura del diseño: es un sistema **read-heavy**. Toda la arquitectura se optimiza para lecturas rápidas con escrituras eventuales.

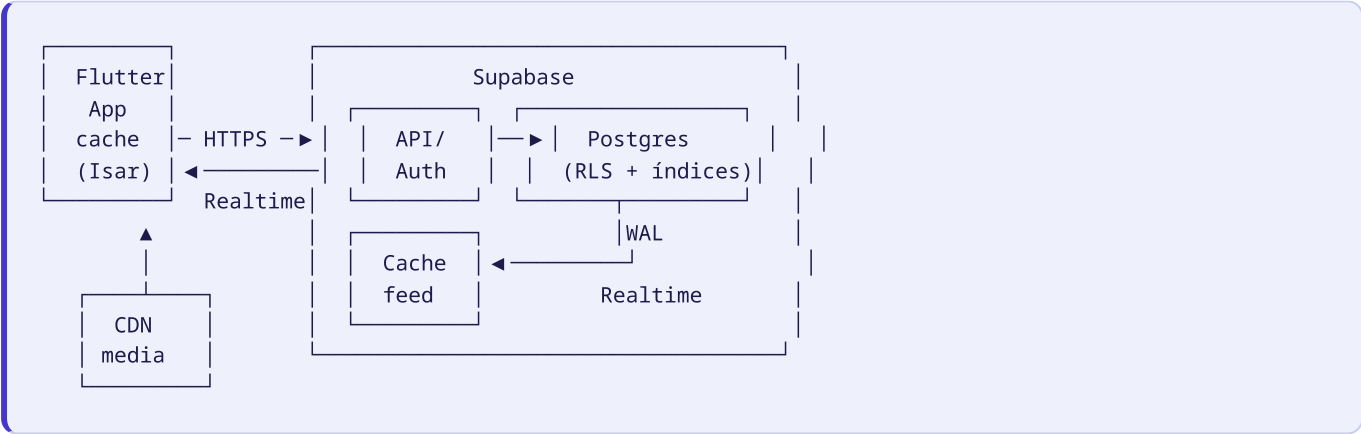
Paso 3: Modelado de datos

Entidades principales:

```
users(id, username, avatar_url)
posts(id, user_id → users, image_url, caption, created_at)
likes(user_id, post_id)          -- tabla puente
```

- **Índices:** `posts(user_id, created_at DESC)`, `likes(post_id)`.
- **RLS:** lecturas públicas (`anon` puede SELECT); escrituras solo dueño (`auth.uid() = user_id`).
- **Elección:** Postgres/SQL por las relaciones `posts↔users↔likes` y transacciones de escritura.

Paso 4-5: Arquitectura de alto nivel



Componentes:

1. **App (Flutter):** cache local Isar de la última página del feed (archivo 10).
2. **Supabase API + Auth:** lectura del feed, publicación, likes.
3. **Postgres:** fuente de verdad; RLS autoriza por fila.
4. **Cache (Redis):** feed ensamblado por usuario.
5. **CDN:** media (imágenes del feed).
6. **Realtime:** (opcional en feed) likes que llegan en vivo.

Paso 6: Decisiones de diseño con trade-offs

Decisión	Por qué	Trade-off aceptado
Cache-aside en el feed (archivo 05)	Lecturas 10:1, feed ensamblado caro	Primer acceso tras expiración es lento
Postgres como fuente de verdad	Consistencia fuerte + RLS nativo	Escala menor que NoSQL puro
CDN para media	Latencia global de imágenes	TTL e invalidación por borrado

Decisión	Por qué	Trade-off aceptado
Offline-first con cache local	App útil sin red (archivo 10)	Sincronización = complejidad extra
Contadores eventuales	Likes no críticos en tiempo real	Valor de likes puede quedar desfasado

El flujo del feed (leer)

1. App pide página de feed
2. ¿Cache local tiene y es fresca? → devuelve (offline)
3. Si no → GET /feed (Supabase)
4. ¿Redis tiene el feed ensamblado? → devuelve
5. Si no → Postgres ensambla joins → se guarda en Redis → devuelve
6. La app guarda en Isar para la próxima

El flujo de publicar (escribir)

1. Usuario publica → insert en Postgres (RLS valida dueño)
2. Se invalida el cache del feed del autor (write-through)
3. Media sube a Storage → CDN invalida la versión vieja
4. Realtime (opcional) avisa a seguidores conectados

Paso 7: Consistencia y disponibilidad

- **Modelo:** fuerte en escritura (Postgres transaccional), eventual en lecturas (cache + Realtime).
 - **SLA:** lectura 99.9%, escritura 99.9% (nueves del archivo 07).
 - **Patrones:** retries + backoff en la app, cache local como fallback sin red.
- Justificación CAP:** el feed prioriza **AP** (siempre responde, eventualmente consistente) porque una foto "unos segundos tarde" no es crítico. El *post* en sí usa strong consistency para no perder publicaciones.

Paso 8-9: Seguridad y observabilidad

- Seguridad:**
- HTTPS en todo, tokens en secure storage.
 - RLS deny-by-default; lecturas públicas solo del feed público.
 - Service role key solo en backend/Edge Functions.

Observabilidad (lo mínimo que mide): | Métrica | Alerta | |---|---| | Error rate del feed > 5% | Revisar query/Edge Function | | Latencia p95 del feed > 500ms | Revisar cache/Redis | | Falla de escrituras | Backoff + alerta | | Cache miss rate alto | Revisar TTL y prefetch |

Paso 10: Riesgos y evolución

Riesgo	Mitigación
Crecimiento a 100M MAU	Replicas de lectura, particionado por usuario
Media ocupa mucho storage/CDN	Compresión, jerarquía de cache media
Feed global (seguir a todos)	Precomputar timeline (fanout on write)
Saturar Postgres con joins	Denormalización + materialized views

Fuentes del caso

- [The System Design Primer — Example: designing a social news feed](#)
 - [ByteByteGo — Design Facebook/Instagram Feed](#)
 - Archivos 02, 04, 05, 06 y 10 de este módulo.
-

Siguiente: [13-caso-chat-mensajeria.md](#)

13: Caso de Estudio — Chat / Mensajería

Aplicación de la plantilla al diseño de un chat en vivo con Supabase Realtime. Los módulos 05 (realtime) y 10 (offline) se unen aquí en una arquitectura concreta.

Paso 1: Alcance y requerimientos

Producto: chat de mensajería 1:1 y grupal.

Incluido	Fuera de scope (este diseño)
Mensajes 1:1 y grupos	Llamadas de voz/video
Realtime (WebSocket)	Encrypted E2E avanzado (clave rotativa)
Offline + cola de envío	Edición/borrado de mensajes
Presencia (online/offline)	Buscador full-text de mensajes

No funcionales: latencia de entrega < 500ms en línea, mensajes **nunca se pierden**, funciona offline y sincroniza al reconectar.

Paso 2: Estimación de escala

Métrica	Valor
Usuarios	5M MAU, 200K DAU
Mensajes por día	~10M
QPS de escritura	~115 msg/s
Conexiones WebSocket activas (pico)	~50K
Storage mensajes/año	~11 GB de texto

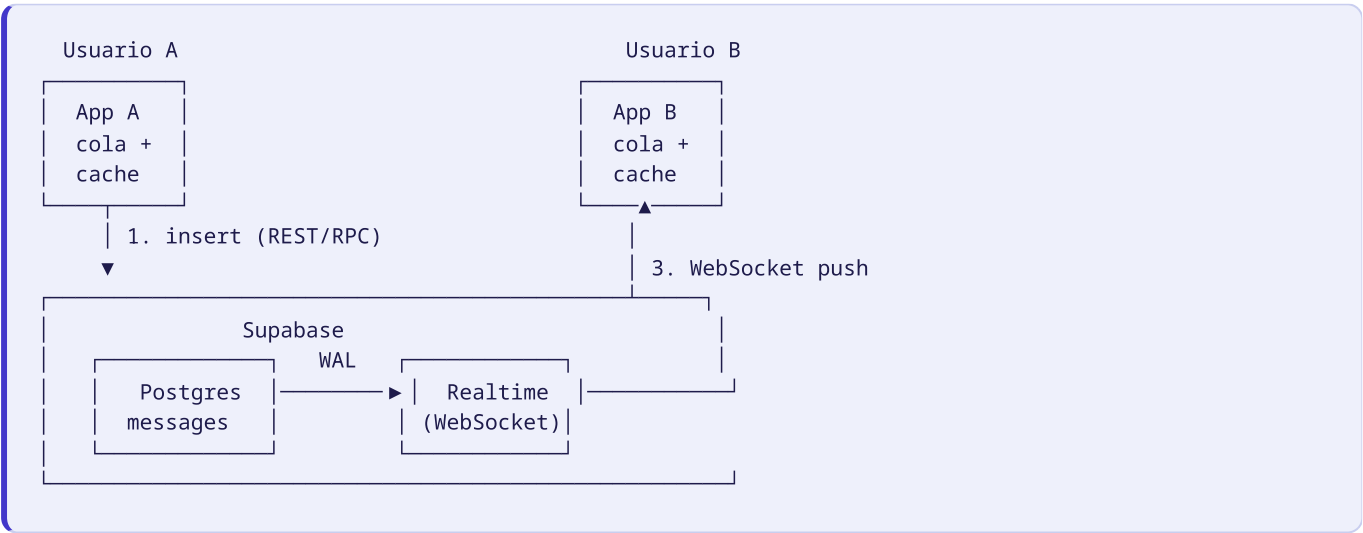
Lectura: es un sistema **write-heavy + realtime**. El cuello de botella es la entrega en vivo, no el almacenamiento.

Paso 3: Modelado de datos

```
conversations(id, type, created_at)
conversation_participants(conversation_id, user_id) -- membresía
messages(id, conversation_id, sender_id, content, created_at)
```

- **Índices:** `messages(conversation_id, created_at)`; `conversation_participants(user_id)`.
- **RLS:** un usuario solo lee mensajes de conversaciones donde **participa** (`auth.uid() IN participant list`).
- **UUIDs como PK** (módulo 03): necesarios para colisiones de offline.

Paso 4-5: Arquitectura de alto nivel



Componentes:

1. **App (Flutter):** cache local (Isar) + cola de escrituras offline.
2. **Supabase Postgres:** fuente de verdad; RLS por participación.
3. **Supabase Realtime:** Postgres Changes + Presence.
4. **API/RPC:** `send_message` como transacción validada.

Paso 6: Decisiones de diseño con trade-offs

Decisión	Por qué	Trade-off aceptado
Realtime/WebSocket (archivo 06)	Latencia < 500ms y bidireccional	Gestión de conexiones, límites
Insert en Postgres como "canal"	Un solo mecanismo de persistencia + entrega	Realtime depende de la DB

Decisión	Por qué	Trade-off aceptado
Offline + cola con optimistic UI	No perder mensajes sin red	Complejidad de sincronización
Presence con Realtime	Saber quién está en línea sin polling	Solo muestra en sesiones activas
UUIDs + <code>updated_at</code>	Sincronización sin colisiones (LWW)	IDs más largos

Flujo de envío de mensaje (online)

1. App muestra el mensaje al instante (optimistic UI) y lo guarda en cola
2. App inserta en Postgres vía RPC (RLS valida participación)
3. Postgres persiste → Realtime detecta el insert por WAL
4. Realtime empuja a todos los suscritos de la conversación (App B)
5. App B recibe y actualiza su UI; App A marca su mensaje como "synced"

Flujo de envío (offline)

1. Sin red: mensaje queda en cola local, marcado "pending"
2. Al reconectar: cola se envía en orden con retries + backoff
3. Éxito → "synced"; fallo → reintentos con límite y alerta

Conflicto (LWW)

Dos devices editando/creando el mismo mensaje → gana el que tenga `updated_at` más reciente. Definido ANTES de implementar (archivo 10).

Paso 7: Consistencia y disponibilidad

- **Modelo:** strong en la entrega de un mensaje (insert transaccional), eventual en el orden percibido por cada cliente (caché local).
- **SLA:** entrega online 99.9%; offline eventual (sin SLA de tiempo).
- **Patrones:** retries + backoff en cola, reconnect automático del WebSocket, cache local como única UI en modo offline.

Justificación CAP: el chat es **AP** en la entrega percibida (siempre responde, orden eventual) pero el mensaje **no se pierde** — la persistencia es fuerte en el servidor.

Paso 8-9: Seguridad y observabilidad

Seguridad:

- RLS: leer mensajes **solo** si participas en la conversación (crítico en chats).
- Presence/Postgres Changes con RLS: no exponer mensajes de chats ajenos.
- HTTPS + tokens en secure storage.

Observabilidad: | Métrica | Alerta | |---|---| | Latencia de entrega > 500ms | Revisar Realtime/DB | | Mensajes en cola creciendo | Revisar conectividad/API | | Error rate de send > 2% | Revisar RPC/RLS | | Conexiones Realtime cercanas al límite | Reducir suscripciones por cliente |

Paso 10: Riesgos y evolución

Riesgo	Mitigación
Escalar WebSockets (500K+ conexiones)	Capa de gateway de websockets / servicios dedicados
Cola offline gigante al reconectar	Sincronizar por lotes + delta sync
Realtime filtra mensajes ajenos	RLS en los canales; test de aislamiento
Orden de mensajes en multi-device	Clocks monotónicos + LWW por <code>updated_at</code>

Fuentes del caso

- [Supabase Docs — Realtime Postgres Changes](#)
- [Supabase Docs — Realtime Presence](#)
- [ByteByteGo — Design a Real-time Chat System](#)
- Archivos 06 y 10 de este módulo.

Siguiente: 14-caso-ecommerce-escalable.md

14: Caso de Estudio — E-Commerce Escalable

Aplicación de la plantilla a un e-commerce: catálogo, carrito, checkout e inventario. Aquí la **consistencia fuerte** importa (dinero y stock), a diferencia de feed y chat.

Paso 1: Alcance y requerimientos

Producto: tienda online con catálogo, carrito y checkout.

Incluido	Fuera de scope (este diseño)
Catálogo + búsqueda + productos	Recomendaciones ML
Carrito (guardado por usuario)	Wishlist, reviews
Checkout + pago (mock)	Envíos/tracking físico, marketplaces
Inventario/stock	Multi-moneda, cupones complejos

No funcionales: lecturas del catálogo muy rápidas; **checkout sin race conditions**; el stock **no puede sobregirar**; alta disponibilidad en ventas.

Paso 2: Estimación de escala

Métrica	Valor
Productos en catálogo	500K
Visitas diarias	100K (10:1 lectura/escritura)
Checkouts/día (pico)	10K
QPS de catálogo	~1.1K
QPS de checkout (pico, Black Friday ×10)	~11 → 115

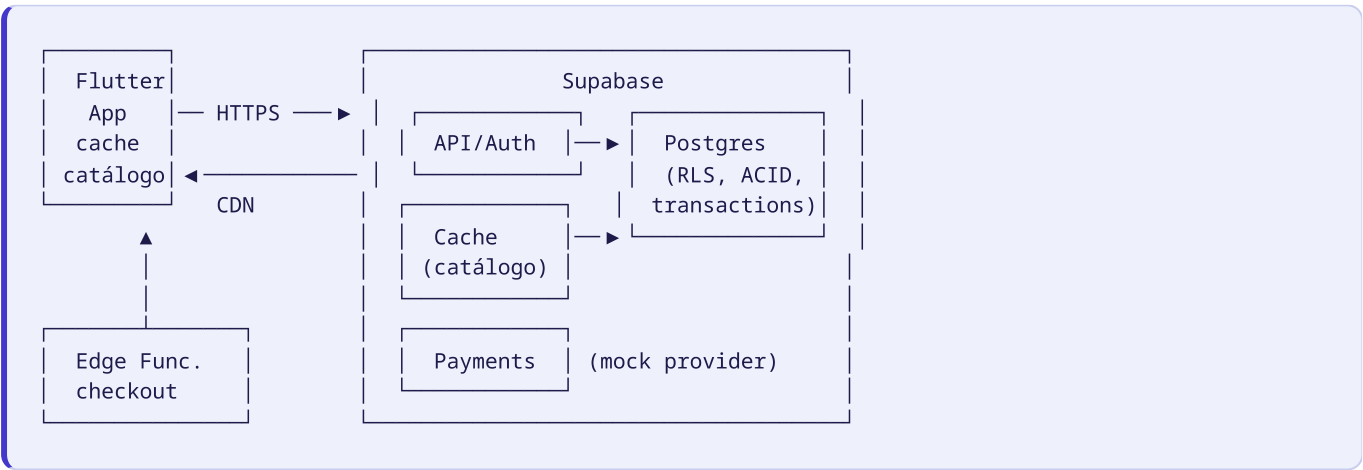
Lectura: catálogo es read-heavy; **checkout es write-heavy crítico** con consistencia fuerte.

Paso 3: Modelado de datos

```
products(id, name, price_cents, stock_available, created_at)
cart_items(user_id, product_id, quantity)
orders(id, user_id, status, total_cents, created_at)
order_items(order_id, product_id, qty, unit_price_cents)
```

- **Índices:** `products` por nombre/categoría; `cart_items(user_id)`; `orders(user_id, created_at)`.
- **RLS:** carrito y órdenes solo del propio usuario; catálogo público.
- **Consistencia del stock:** actualizar stock **dentro de la misma transacción** del checkout, no de forma asíncrona.

Paso 4-5: Arquitectura de alto nivel



Componentes:

1. **App:** cache del catálogo (lee rápido, offline del catálogo).
2. **CDN:** imágenes de productos.
3. **Cache:** catálogo/búsqueda.
4. **Postgres:** carrito, órdenes, stock — con transacciones.
5. **Edge Function (checkout):** lógica crítica con pago.

Paso 6: Decisiones de diseño con trade-offs

Decisión	Por qué	Trade-off aceptado
Catálogo con cache-aside	Lecturas 10:1, sin datos críticos	Stock visible puede estar desfasado (mostrar "últimas unidades")

Decisión	Por qué	Trade-off aceptado
Checkout transaccional (ACID)	Dinero y stock no admiten eventual	Checkout es más lento que el resto
Edge Function para checkout	Lógica crítica aislada + pago	Complejidad de despliegue
Stock con cola asíncrona (para cupos)	Ventas masivas	Backpressure + monitoreo de cola
RLS por usuario en carrito/órdenes	Aislamiento de datos	Nada accesible vía API directa

Flujo del checkout (sin race condition)

1. App envía orden a la Edge Function
2. Edge Function abre transacción en Postgres:
 - a. SELECT stock FOR UPDATE del producto ← evita oversell
 - b. valida stock >= qty; si no → error "sin stock"
 - c. resta stock, inserta order + order_items
 - d. COMMIT
3. Se llama al provider de pago (mock) → estado "paid"
4. Se invalida el cache del producto (stock nuevo)

El **FOR UPDATE** (locking) dentro de la transacción es lo que **previene dos compradores** de llevarse la última unidad simultáneamente.

Paso 7: Consistencia y disponibilidad

- **Modelo:** strong consistency en checkout (ACID), eventual en catálogo (cache).
- **SLA:** checkout 99.99% (dinero en juego); catálogo 99.9%.
- **Patrones:** idempotencia en pagos (mismo order id no paga dos veces), retries con backoff en la app, fallback de catálogo con cache local.

Justificación CAP: el checkout elige **CP** (prefiere rechazar una venta antes que dejar pasar un oversell); el catálogo elige **AP** (siempre muestra productos, aunque el stock exacto sea eventual).

Paso 8-9: Seguridad y observabilidad

Seguridad:

- Precios/total **nunca vienen de la app** (se calculan en el servidor).
- Idempotencia: el cliente no puede duplicar cargos.
- RLS por usuario; service role solo en la Edge Function.
- Cifrado de datos de pago (PII) — PCI-DSS si es producción real.

Observabilidad: | Métrica | Alerta | |---|---| | Tasa de checkout fallidos > 1% | Revisar transacciones/pago |
| Stock en 0 con pedidos fallidos | Revisar locking y cola | | Latencia de checkout p95 > 2s | Revisar Edge
Function/DB | | Errores de pago | Revisar provider + idempotencia |

Paso 10: Riesgos y evolución

Riesgo	Mitigación
Oversell en pico	Transacciones + FOR UPDATE + verificación final
Cache del stock desactualizado	Mostrar "pocas unidades" + invalidación al vender
Pago fallido a mitad del checkout	Estados de orden (pending/paid/failed) + reintento idempotente
Escalar checkout	Particionar órdenes por usuario; colas por lote

Fuentes del caso

- [ByteByteGo — How to Design an E-commerce System](#)
- [ByteByteGo — How to Design a Flash Sale System](#)
- [Supabase Docs — PostgreSQL Transactions](#)
- [Designing Data-Intensive Applications — M. Kleppmann](#) (transacciones y aislamiento)

Siguiente: 15-caso-saas-multi-tenant.md

15: Caso de Estudio — SaaS Multi-Tenant

Aplicación de la plantilla a una plataforma SaaS donde muchos clientes (tenants) comparten la misma infraestructura. El aislamiento entre tenants es el problema de diseño central.

Paso 1: Alcance y requerimientos

Producto: SaaS de gestión de proyectos (tipo Notion/Trello) con múltiples organizaciones.

Incluido	Fuera de scope (este diseño)
Organizaciones + membresías	Billing/planes reales
Proyectos y tareas por org	Integraciones de terceros
Permisos por rol (admin/member/viewer)	Multi-region
Uso compartido de infraestructura	White-labeling

No funcionales: aislamiento **estricto** entre tenants (un tenant no puede ver datos de otro), escala de miles de tenants, disponibilidad alta.

Paso 2: Estimación de escala

Métrica	Valor
Tenants activos	5K
Usuarios totales	500K (100 por tenant)
Filas de tareas	50M
QPS total	~1K

Lectura: la complejidad no está en la escala bruta, sino en **cómo aislar 5K tenants** dentro de una sola base de datos.

Paso 3: Modelado de datos (el corazón del SaaS)

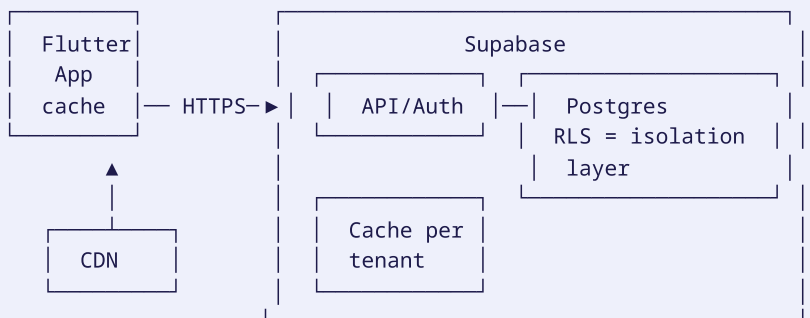
```
organizations(id, name, plan)
organization_members(org_id, user_id, role)  -- 'admin' | 'member' | 'viewer'
projects(id, org_id, name, created_at)
tasks(id, project_id, title, status, assignee_id, created_at)
```

- **Índices:** `projects(org_id)`, `tasks(project_id)`, `organization_members(user_id)`.
- **RLS (lo más importante):** toda consulta se filtra por membresía. Un usuario solo ve tareas si es miembro de la org del proyecto.

```
CREATE POLICY "read_projects_of_my_orgs" ON projects
FOR SELECT USING (
  EXISTS (
    SELECT 1 FROM organization_members m
    WHERE m.org_id = projects.org_id
    AND m.user_id = auth.uid()
  )
);
```

- **Multiplicador de aislamiento:** el mismo patrón se replica en `tasks`, `comments`, etc. Cada tabla con FK a `projects` / `orgs` requiere su política.

Paso 4-5: Arquitectura de alto nivel



Componentes:

1. **App:** cache local por tenant (el cache de un tenant nunca contamina otro).
2. **Supabase Auth + API:** identidad y requests.
3. **Postgres con RLS:** el aislador de tenants (deny-by-default + políticas por membresía).
4. **Cache por tenant:** claves namespaced por `org_id`.

Paso 6: Decisiones de diseño con trade-offs

Decisión	Por qué	Trade-off aceptado
Single-tenant DB + RLS	Un solo Postgres para todos (barato, simple)	Políticas RLS complejas; riesgo si una política falla
RLS como único control de aislamiento	Se evalúa en el servidor; la app no puede saltárselo	Todo nuevo acceso requiere su política
Roles por membresía (admin/member/viewer)	Permisos por fila, no por app	Políticas por rol = más SQL
Cache namespaced por tenant	No filtrar datos entre tenants	Invalidate por tenant al escribir
Row Level Security test en CI	Prevenir fuga entre tenants	Test suite dedicada de aislamiento

El patrón de aislamiento que se repite

1. El usuario se autentica → JWT con `auth.uid()`
2. Cada query pasa por RLS → solo filas cuya org lo incluye como miembro
3. Los permisos por rol (admin puede borrar, viewer solo lee) se codifican en políticas RLS adicionales
4. Ninguna query en la app usa el service role (se saltaría RLS)

Paso 7: Consistencia y disponibilidad

- **Modelo:** strong consistency (un Postgres transaccional para todos los tenants).
 - **SLA:** 99.9% (los tenants dependen del SaaS).
 - **Patrones:** retries/backoff en la app, cache local como fallback de lectura.
- Justificación CAP:** es un sistema **CP por defecto** — la consistencia de datos de proyectos importa más que servir data eventual. La alta disponibilidad se consigue con la infraestructura gestionada de Supabase, no con consistencia eventual.

Paso 8-9: Seguridad y observabilidad

Seguridad (crítica en multi-tenant):

- RLS deny-by-default en **todas** las tablas con datos de tenant.
- Test de aislamiento: simular 2 usuarios de orgs distintas y verificar que **no** se cruzan datos.
- Service role key nunca en la app (rompería el aislamiento).
- Auditoría de quién accede a qué org.

Observabilidad: | Métrica | Alerta | |---|---| | Fuga potencial de RLS (test en CI falla) | Bloquear merge | | Latencia por tenant (un tenant pesado) | Revisar "noisy neighbor" | | Queries sin RLS detectadas | Logs de plan de ejecución | | Cache invalidado por tenant | Monitorear miss rate por org |

Paso 10: Riesgos y evolución

Riesgo	Mitigación
Un tenant satura la DB (noisy neighbor)	Cuotas por tenant, pooling, rate limits
RLS mal escrita filtra datos	Test de aislamiento automatizado en CI
Escalar a miles de tenants pesados	Schema-per-tenant o database-per-tenant (extremos)
Roles complejos	Tabla de permisos + policies parametrizadas

Fuentes del caso

- [Supabase Docs — Row Level Security \(multi-tenant section\)](#)
- [Supabase Docs — Making RLS Easier \(multi-tenant patterns\)](#)
- [ByteByteGo — Multi-tenant architecture](#)
- [Martin Fowler — MultiTenant](#)

Siguiente: 16-ejercicios-practica.md

16: Ejercicios de Práctica

Ejercicios progresivos para interiorizar el system design: de problemas cortos a casos completos con la plantilla del archivo 11.

Nivel 1: Conceptos rápidos (5 min c/u)

1. Explica **latencia vs throughput** con un ejemplo de una cafetería.
 2. Dada una tabla de `posts` con 50M de filas, ¿qué dos índices crearías primero y por qué?
 3. ¿Cuándo elegirías **SQL** sobre **NoSQL**? Da un caso donde NoSQL gane.
 4. Dibuja de memoria las diferencias entre **master-slave** y **master-master**.
 5. ¿Qué estrategia de caché usarías para un feed y por qué?
 6. Explica **CAP** con las letras y un ejemplo de cada par.
 7. ¿Qué es el *cache stampede* y cómo lo mitigas?
 8. Diferencia **autenticación** vs **autorización** con un ejemplo de RLS.
 9. ¿Cuándo usarías **long polling** en vez de **WebSocket**?
 10. Define los 3 pilares de la observabilidad y qué pregunta responde cada uno.
-

Nivel 2: Decisiones con trade-offs (10 min c/u)

1. **Feed**: ¿precomputar el timeline (fanout on write) o ensamblar al leer (fanout on read)? ¿Qué métricas te harían cambiar de opción?
 2. **Chat**: ¿guardar mensajes en Postgres y emitir por Realtime, o usar un message broker? ¿Dónde está la fuente de verdad?
 3. **E-commerce**: un producto con 1 unidad en stock. Dos usuarios compran a la vez. Diseña la secuencia que evita el oversell.
 4. **SaaS**: si un tenant empieza a hacer queries que saturan la DB, ¿qué 3 medidas aplicas y en qué orden?
 5. **Offline**: diseña la cola de sincronización de likes. ¿Cómo evitas re-envíos duplicados?
-

Nivel 3: Mini-casos (15-20 min c/u) — usa la plantilla del archivo 11

Caso A: App de recordatorios con cuentas

- 1M usuarios, notificaciones push, sincronización multi-device.
- **Extras:** latencia de push < 30s, offline obligatorio, RLS para aislamiento por usuario.

Caso B: Plataforma de encuestas en vivo

- Encuestas con resultados en tiempo real mientras el público vota.
- **Extras:** 100K votos en 10 min, resultados via realtime, sin sobrecargar la DB con writes individuales (agrega en batch).

Caso C: SaaS de inventario multi-tenant

- Múltiples comercios gestionan stock y órdenes.
 - **Extras:** aislamiento estricto entre tenants, inventario con consistencia fuerte, reportes (read-heavy).
-

Nivel 4: Casos completos (45-60 min) — con checklist

Diseña (plantilla completa del archivo 11) uno de estos:

1. Instagram/feed con videos cortos

Incluye: feed, likes, comentarios, media. Extras: ranking del feed, recomendación, offline.

2. WhatsApp/Telegram

Incluye: 1:1, grupos, realtime, offline, presencia. Extras: edición de mensajes, media en mensajes.

3. E-commerce con flash sales

Incluye: catálogo, carrito, checkout, stock. Extras: 10× tráfico en minutos, colas, idempotencia.

4. Notion/Trello multi-tenant

Incluye: orgs, permisos por rol, proyectos. Extras: aislamiento RLS, auditoría, migraciones de esquema.

Checklist obligatorio:

- ☐ Alcance + fuera de scope
- ☐ QPS, storage, ratios estimados
- ☐ Diagrama de alto nivel
- ☐ Modelo de datos con índices y RLS
- ☐ 3+ decisiones con trade-offs

- ☐ Consistencia/disponibilidad (CAP) declarada
 - ☐ Seguridad y observabilidad
 - ☐ 3 riesgos + mitigaciones
-

Nivel 5: Auto-entrevista (modo entrevistador)

Graba tu respuesta de 45 min a un caso y evalúate:

1. ¿Clarifiqué el alcance antes de diseñar? ☒/☒
2. ¿Estimé números antes de dibujar? ☒/☒
3. ¿Justifiqué cada decisión con un trade-off? ☒/☒
4. ¿Declaré CAP/consistencia explícitamente? ☒/☒
5. ¿Mencioné seguridad y observabilidad sin que me lo pidan? ☒/☒
6. ¿Cerré con riesgos y evolución? ☒/☒

Métricas de mejora: por cada caso resuelto, intenta reducir el tiempo de los pasos 1-2 a < 5 min (son los que más se repiten en entrevistas).

Fuentes para corregir

- [The System Design Primer — Solutions](#)
 - [System Design Interview — Alex Xu](#)
 - [ByteByteGo — System Design interview problems](#)
-

Siguiente: [17-recursos-externos.md](#)

17: Recursos Externos

Los recursos que se usaron y los que sirven para profundizar. Nada de "listas interminables": cada enlace tiene un **cuándo usarlo**.

Fundamentales (estudiar primero)

Recurso	Qué aporta	Cuándo
The System Design Primer	Base teórica completa + casos resueltos	Lee toda la sección de topics; es la columna del curso
Designing Data-Intensive Applications (Kleppmann)	El libro de referencia de datos, consistencia y replicación	Después de dominar el primer
System Design Interview — Alex Xu	Framework de entrevista y casos paso a paso	Cuando prepares entrevistas

Guías cortas y visuales (para consultar)

Recurso	Cuándo
ByteByteGo — Guides	Diagramas rápidos por tema (cache, chat, e-commerce, multi-tenant)
ByteByteGo — CAP Theorem	Si CAP te confunde (a todos nos pasa)
ByteByteGo — Caching strategies	Cuando dudes entre cache-aside/write-through
Brendan Gregg — USE Method	Para métricas de rendimiento
AWS Architecture Center	Casos reales de arquitecturas cloud

Supabase (el stack del repositorio)

Recurso	Cuándo
Supabase Docs — Database	RLS, índices, transacciones en Postgres

Recurso	Cuándo
Supabase Docs — Realtime	Postgres Changes, Presence, Broadcast
Supabase Docs — Auth	JWT, authn/authz, magic links
Supabase Docs — Edge Functions	Lógica de backend asíncrona (colas, checkout)
Supabase Docs — Logging/Metrics	Observabilidad gestionada
Supabase — The Next.js Starter	Ejemplos oficiales multi-caso

Simuladores / práctica activa

Recurso	Qué hacer
System Design Primer — Solutions	Corregir los ejercicios del archivo 16
ByteByteGo — Interview problems	Casos extra para el Nivel 4
Kleppmann's blog	Profundización en consistencia y streaming

Orden de estudio sugerido

- | | |
|----------------------------------|-------------------------------------|
| 1. System Design Primer (topics) | → teoría base |
| 2. Archivos 00-11 de este módulo | → teoría aplicada al stack |
| 3. Casos 12-15 (plantilla) | → ver el framework en acción |
| 4. Ejercicios 16 | → practicar |
| 5. DDIA / Alex Xu | → profundizar y preparar entrevista |
| 6. Supabase docs | → cuando implementes |

Regla de oro

No leas 10 recursos a la vez. **Uno a la vez**, con el objetivo de resolver el siguiente ejercicio. El System Design Primer es el único que vale leer de principio a fin; el resto es consulta puntual.

Siguiente: BIBLIOGRAFIA.md

Bibliografía

Fuentes principales de este módulo, con el motivo de su inclusión. Formato: autor — título — motivo.

Libros

- **Martin Kleppmann** — *Designing Data-Intensive Applications* — La referencia definitiva sobre datos distribuidos: replicación, particionado, transacciones, consistencia. Base teórica de los archivos 04, 07 y 14.
 - **Alex Xu** — *System Design Interview (Vol. 1 y 2)* — Framework de entrevistas y casos resueltos paso a paso. Base de la plantilla del archivo 11 y del Nivel 4 del archivo 16.
 - **Donne Martin** — *The System Design Primer* — Compendio gratuito de todo lo que un system designer necesita: conceptos, trade-offs, casos. Es la columna vertebral del curso (usado como fuente primaria en 00-10).
 - **Brendan Gregg** — *Systems Performance* — Métricas y metodología USE para diagnóstico de rendimiento. Referencia puntual del archivo 09.
-

Documentación oficial

- **Supabase Docs** — *Database / Realtime / Auth / Functions / Telemetry* — Documentación del stack del repositorio; usada para RLS (03), Realtime (06), seguridad (08) y observabilidad (09).
 - **Saltzer & Schroeder** — *The Protection of Information in Computer Systems* — Paper clásico del principio de menor privilegio (archivo 08).
 - **OWASP** — *OWASP Top 10* — Marco de riesgos de seguridad en aplicaciones web (archivo 08).
-

Artículos y guías en línea

- **ByteByteGo** — *Caching strategies, CAP theorem, Short/long polling vs SSE vs WebSocket, Timeouts & Retries, Observability, Chat/E-commerce/Multi-tenant designs* — Diagramas y explicaciones concisas que sustentan los archivos 05, 06, 07, 09, 12, 13, 14 y 15.
 - **Brendan Gregg** — *The USE Method* — Guía corta de métricas de rendimiento (archivo 09).
-

Criterio de inclusión

1. Todo lo citado se **usó** en algún archivo de este módulo (nada decorativo).
2. Se priorizaron fuentes **libres/gratuitas** (System Design Primer, docs oficiales, ByteByteGo, papers clásicos).
3. Los libros (Kleppmann, Xu) aparecen como **profundización**, no como requisito para empezar.